CobGDB - GnuCOBOL DEBUGGER

Table of Contents

1. Introduction....2

1.1. Installing the debugger CobGDB on Windows....3
1.2. Compile and Debug GnuCOBOL programs....4
1.3. Main Commands....5

2. Tutorial - Sample Debugging Session....6

2.1. ? - Help Command....8
2.2. R - Run Command....9
2.3. S - Step Command....11
2.4. G - Go Command....13
2.5. D - Display Variables Command....14
2.6. H - Show Command....15
2.6.1. E - Edit subCommand....16
2.7. V - Variables Command....17
2.7.1. Enter - subCommand....18
2.7.2. E - Edit subCommand....19
2.7.3. R - Return subCommand....20
2.8. S - Step Command....21
2.9. O - Focus Command....23
2.10. Pop-up Variable windows....25
2.11. F - File Command....26
2.12. R - Run Command....29
2.13. W - Window Size command....30
2.14. Q - Quit Command....33
2.15. A - Attach Command....34

3.Other Commands....36

3.1.CTRL-A - Autostep command....36
3.2.CTRL-F - Find command....37
3.3. Change the Current Line background color....38
3.4. Display Variable content by hover the mouse....38
3.5. CTRL-S and CTRL-B - Save and Load Breakpoints....39
3.6. Debugging a pre-compiled Program....40
3.7. Debugging sub programs....41
3.8. CobGDB Version....44

4. Document Change Log....45

1. Introduction

CobGDB is a TUI (Text User Interface) application, programmed in C, designed to assist in animate and debugging GnuCOBOL programs. It uses **GDB** (the GNU Debugger at https://www.gnu.org/software/gdb/).

The CobGDB project is hosted at ***https://github.com/marcsosduma/cobgdb***

You don't need to know the GDB product and its many commands (https://www.sourceware.org/gdb/). ***CobGDB has its own user interface (described in this document) that is very simple to use and is responsible for interfacing the underlying GDB which is the real debug and animate engine but operates practically in a transparent way to the GnuCOBOL developer.***
At https://github.com/marcsosduma/cobgdb/tree/main/windows, the executable program cobgdb.exe for this operating system is available and ready to use.

To compile CobGDB from C source code on Windows, you can use MinGW.
The Makefile is configured to generate the program **`cobgdb.exe`** for both Windows and Linux.

The idea of CobGDB application is based on the extension for Visual Studio Code (VSCode) created by Oleg Kunitsyn, which can be found on GitHub: *https://github.com/OlegKunitsyn/gnucobol-debug*.

You can find an updated copy of this document at: https://github.com/marcsosduma/cobgdb/tree/main/doc.

1.1. Installing the debugger CobGDB on Windows

On Windows, just download **`cobgdb.exe`** from following folder:
https://github.com/marcsosduma/cobgdb/tree/main/windows .

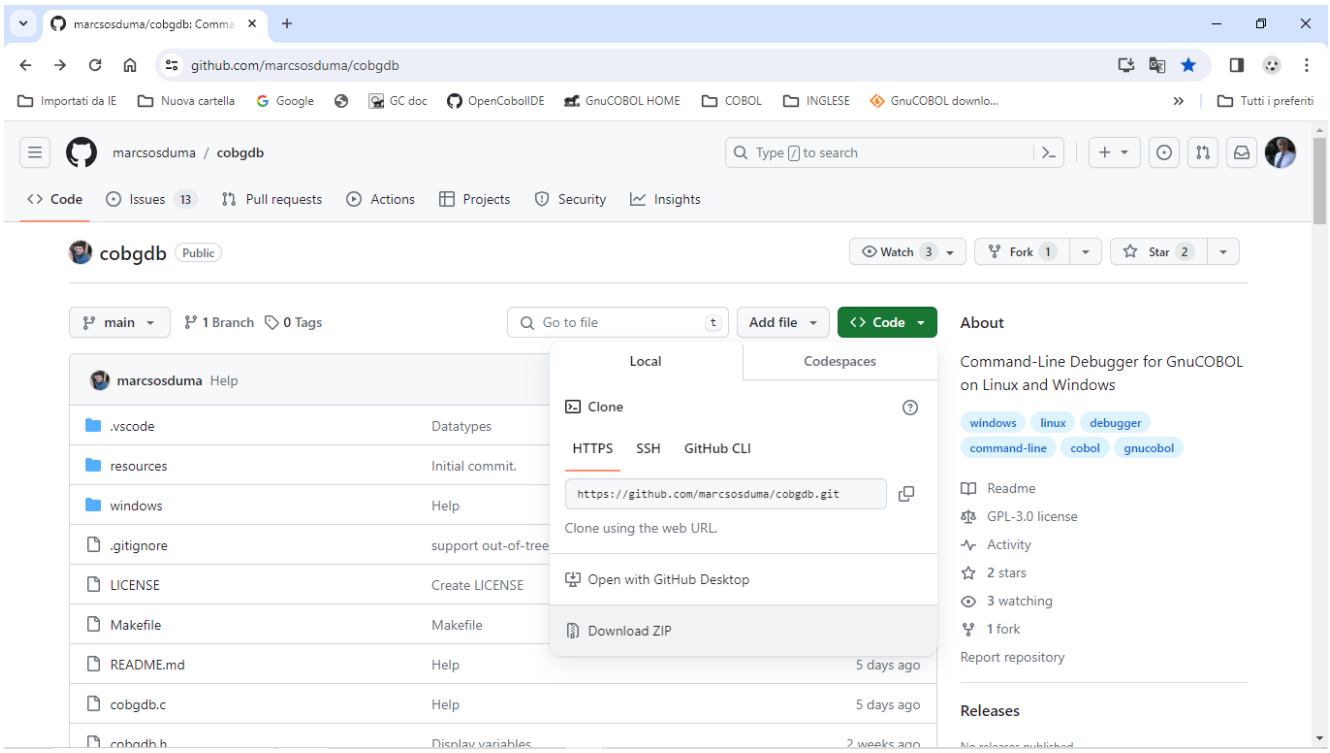
As an example you can put **`cobgdb.exe`** into the "**`bin`**" folder of your GnuCOBOL installation (the same folder where the GnuCOBOL compiler **`cobc.exe`** is located)

or

first install MinGW (Minimalist GNU for Windows).
Then execute the make ('mingw32-make' for Windows) command to compile the code from C source.

Note: if you have security problems downloading an .exe file then you can try download the entire repository with the following github button < > Code ▼ --> *Download ZIP*

Then unzip the file and copy cobgdb.exe to the "bin" folder of your GnuCOBOL installation (the same folder where the GnuCOBOL compiler cobc.exe is located)



1.2. Compile and Debug GnuCOBOL programs

Compile and run a debugging session of the sample program using the following command:
`cobgdb customer.cob -x`

Source code of **`customer.cob`** sample program used also for following tutorial is at:
https://github.com/marcsosduma/cobgdb/tree/main/windows

or, other example for **`cobc`** parameters , use :
`cobgdb customer.cob -x -Tcustomer.txt`

where '**`-Tcustomer.txt`**' is an argument that can be indirectly passed to '**`cobc`**' by '**`cobgdb,`**' even if it is not used by '**`cobgdb`**' itself. (**`-T`** just creates a compilation list output into the **`customer.txt`** file)

CobGDB takes one or more programs with COB or CBL extension as parameters and runs the GnuCOBOL compiler with the following format:
`cobc -g -fsource-location -ftraceall -v -free -O0 -x prog.cob prog2.cob ...`

note:
-g as we know, it is used to add “debug symbols” (variable names, variable types, correspondence between assembler binary instructions and source code lines) inside the executable code (inside the .exe or .dll file).
-O0 is used to tell the compiler not to optimize, for example it must also compile parts of the source code that would never be executed, otherwise the debugger will not find an exact correspondence between the executable and the source lines.

To debug multiple programs, use COBGDB with the following syntax :
`cobgdb prog.cob subprog1.cob subprog2.cob ...`

This will create a single **`prog.exe`** executable to debug.
To debug sub programs separately , see also "Debugging sub Programs" chapter in this document.

You can also run GDB/GDBSERVER remotely using the "A" (Attach) key.
CobGDB will prompt you to provide the server and port in the format server: port or the PID of the application.
Example:
- localhost: 5555
- 9112

1.3. Main Commands

Command	Short	Description
?	Help	Show the HELP window
R	Run	Start the program and Run until a breakpoint is encountered, also when, by default, the first Breakpoint is at first program statement. Always use this command to start a debugging session. If a debugging session is active "R" will re-start the program from the beginning.
B	Breakpoint	Set or Unset a Breakpoint at current line of the Procedure Division code.
S	Step (Step Into)	Runs the program until the next line. If needed it goes into a subroutine executed by PERFORM or CALL.
N	Next (Step Over)	Runs the program until the next line but does not enter a subroutine executed by PERFORM or CALL.
G	Go (Step Out)	Executes the remaining statements of the current subroutine and returns to the calling point. Continues the program execution until return from a subroutine (a PERFORM or a CALL, same as a Step Out command) or until encounters a stopping point (a Breakpoint or the end of the program).
C	Cursor	Runs the program until it reaches the selected line at Cursor location.
J	Jump	Ask for a line number and runs the program until it reaches that line.
V	Variables	Displays a window with a list of all variables and their content for the running program. From this window you can also change the content of variables.
H	sHow	Displays a window with a list of variables and its content from the cursor selected line. From this window you can also edit / change the content of variables from the selected line. Right-click on a source line is same as command "H".
D	Display	Set to OFF or ON the automatic Display of variables of current and previous statement in execution during a debugging session. At program start it is OFF.
O	fOcus	When program encounters an ACCEPT verb, this command switch the focus from the debugging screen to the application screen to accept the user input and then automatic go back to the debugging screen.
F	File	When CobGDB is executed with more than one program, allows selecting one of those source file to manage debugging commands (es. Setting new Breakpoints).
A	Attach	Attach to GDBSERVER or to an Application PID.
W	Window	Switch between two window size: 24 rows x 80 cols or 34 rows x 132 cols.
CTRL-A	Autostep	Automatically executes the "Step" command until next breakpoint or press ESC.
CTRL-F	Find text	Ask for some text and search in the source code.
CTRL-G	Go to line	Ask for a line number and move the current/cursor line in the source code.
CTRL-S	Save breakpoints	Save all Breakpoints from the current debugging session.
CTRL-B	Load breakpoints	Load all saved Breakpoints from a previous debugging session.
+ or **-**		Change the background color of the current line / cursor line
Q	Quit	Quits (ends) the debugger and the program (or programs).

2. Tutorial - Sample Debugging Session

This tutorial is on a Windows 10 or Windows 11 platform using following version of GnuCOBOL:

```
cobc (GnuCOBOL) 3.2.0
Copyright (C) 2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
Written by Keisuke Nishida, Roger While, Ron Norman, Simon Sobisch, Edward Hart
Built Jul 28 2023 16:07:38
Packaged Jul 28 2023 16:58:47 UTC
C version (MinGW) "13.1.0"
```

Downloaded from *https://www.arnoldtrembley.com/GnuCOBOL.htm*

====== Version 3.2 ======
GnuCOBOL 3.2 (28Jul2023) MSYS2 64-bit GC32M-BDB-x64.7z -- MSYS2 64-bit GnuCOBOL 3.2 Final release **with full debugging support**. (95.4 Megabytes).
GnuCOBOL 3.2 (28Jul2023) MSYS2 32-bit GC32M-BDB-x32.7z -- MSYS2 32-bit GnuCOBOL 3.2 Final release **with full debugging support**. (96.1 Megabytes).

After executing **cobgdb customer.cob -x** the application automatically inserts a Breakpoint at first executable program statement of PROCEDURE DIVISON (see the B symbol at left of line 103 in this sample) and displays following screen:

```
Prompt dei comandi - cobgdb  customer.cob
CobGDB v2.3.0                GnuCOBOL Debugger                ► → ↓ ↑ ■ ↔ D ?
  100
  101
  102         PROCEDURE DIVISION.
B 103         001-START.
  104             SET ENVIRONMENT 'COB_SCREEN_EXCEPTIONS' TO 'Y'
  105             SET ENVIRONMENT 'COB_SCREEN_ESC' TO 'Y'
  106             SET ENVIRONMENT 'ESCDELAY' TO '25'
  107             *>CALL "SYSTEM" USING "chcp 437" WS-STATUS
  108             *>CALL "SYSTEM" USING "mode con: lines=24 cols=80" *> WS-STATU
  109             ACCEPT WS_NUMR FROM LINES
  110             ACCEPT WS-NUMC012 FROM COLUMNS *> WS-STATUS
  111             PERFORM 007-OPEN-FILES
  112             PERFORM UNTIL E-EXIT
  113                 MOVE "MENU" TO WS-OP
  114                 MOVE "CHOOSE AN OPTION" TO WS-STATUS  *> WS-STATUS
  115                 MOVE SPACES TO WS-CHOICE
  116                 DISPLAY SS-CLS
  117                 ACCEPT SS-MENU
  118
  119                 EVALUATE TRUE
  120                     WHEN E-INCLUDE
c:/GC-AWORK/customer.cob
```

You can scroll the source code window with UP or DOWN keys , with PG UP or PG DOWN keys , with mouse left click on the ▲ and ▼ symbols or on the scroll bar, dragging the slide bar present in the scroll bar.or using the mouse wheel .Use RIGHT key and LEFT key to scroll horizontally.

In the upper right window corner there is the "Symbol bar" where you can find following symbols:

► ➡ ⬇ ⬆ ■ ↔ D ?

	Symbol	Command	
	►	**Run**	Start (or Re-Start) the debugging session
	➡	**Next**	Run until next statement and do not enter Call or Perform routine ("***Step Over***")
	⬇	**Step**	Run until next statement and enter Call or Perform routine ("***Step Into***")
	⬆	**Go**	Run until next stopping point, (es.exit from Call or Perform routine as "***Step Out***")
	■	**Quit**	Ends the debugging session
	↔	**Focus**	Switch the focus to the application window and back
	D	**Display**	Toggle on / off the automatic variable display during animation
	?	**Help**	Displays the Help window

when you hover over one of these symbol, you get the corresponding command description (like a tooltip) displayed at the bottom left of the screen.

2.1. ? - Help Command

Type **?** (key) HELP command or left click with mouse on the **?** symbol:

```
Prompt dei comandi - q3
CobGDB                    GnuCOBOL GDB Interpreter
 100                10 COLUMN PLUS 2 TO WS-ERROR.
 101
 102         PROCEDURE DIVISION.
B 103         001-START.
 104             SET ENVIRONMENT 'COB_SCREEN_EXCEPTIONS' TO 'Y'
 105             SET ENVIRONMENT 'COB_SCREEN_ESC' TO 'Y'
 106             SET ENVIRONMENT 'ESCDELAY' TO '25'
 107             *>CALL "SYSTEM" USING "chcp 437" WS-STATUS
 108             *>CALL "SYSTEM" USING "mode con: lines=24 cols=80" *> WS-STATU
 109             ACCEPT WS_NUMR FROM LINES
 110             ACCEPT WS-NUMC012 FROM COLUMNS *> WS-STATUS
 111             PERFORM 007-OPEN-FILES
 112             PERFORM UNTIL E-EXIT
 113                 MOVE "MENU" TO WS-OP
 114                 MOVE "CHOOSE AN OPTION" TO WS-STATUS  *> WS-STATUS
 115                 MOVE SPACES TO WS-CHOICE
 116                 DISPLAY SS-CLS
 117                 ACCEPT SS-MENU
 118
 119                 EVALUATE TRUE
 120                     WHEN E-INCLUDE
C:/GC-AWORK/customer.cob
help
```

the HELP window is displayed

```
Prompt dei comandi - q3
CobGDB                    GnuCOBOL GDB Interpreter
 100                10 COLUMN PLUS 2 TO WS-ERROR.
 101
 10 CobGDB HELP
B 10                    CobGDB - Commands
 10
 10 B - Breakpoint: toggles the breakpoint at the current selected line
 10     (can also be done with the mouse).
 10 R - Run: runs the program from the first statement until a breakpoint
 10     is encountered.                                                  ATU
 10 C - Cursor or Continue: runs the program until it reaches the
 11     selected line.
 11 J - Jump: runs the program until it reaches the specified line.
 11 N - Next: runs the program until the next line but does not enter
 11     a subroutine executed by CALL or PERFORM.
 11 S - Step: runs the program until the next line.
 11 G - GO: continues the program execution until it encounters a
 11     stopping point: breakpoint, end of the program, or the
 11     return from a subroutine - PERFORM/CALL.
 11
 119                 EVALUATE TRUE
 120                     WHEN E-INCLUDE
C:/GC-AWORK/customer.cob
help
```

scroll the Help window with cursor keys UP and DOWN or mouse wheel.
Use ESC or Enter or left click to exit from this HELP window and return to debugging session.

2.2. R - Run Command

To start executing the program and the debugging session from first program statement you always have to use the "**R**" command (key) or left click with mouse on the ► **Run** symbol

```
Prompt dei comandi - q3
CobGDB                    GnuCOBOL GDB Interpreter
 100                10 COLUMN PLUS 2 TO WS-ERROR.
 101
 102         PROCEDURE DIVISION.
B 103         001-START.
 104             SET ENVIRONMENT 'COB_SCREEN_EXCEPTIONS' TO 'Y'
 105             SET ENVIRONMENT 'COB_SCREEN_ESC' TO 'Y'
 106             SET ENVIRONMENT 'ESCDELAY' TO '25'
 107             *>CALL "SYSTEM" USING "chcp 437" WS-STATUS
 108             *>CALL "SYSTEM" USING "mode con: lines=24 cols=80" *> WS-STATU
 109             ACCEPT WS_NUMR FROM LINES
 110             ACCEPT WS-NUMC012 FROM COLUMNS *> WS-STATUS
 111             PERFORM 007-OPEN-FILES
 112             PERFORM UNTIL E-EXIT
 113                 MOVE "MENU" TO WS-OP
 114                 MOVE "CHOOSE AN OPTION" TO WS-STATUS  *> WS-STATUS
 115                 MOVE SPACES TO WS-CHOICE
 116                 DISPLAY SS-CLS
 117                 ACCEPT SS-MENU
 118
 119                 EVALUATE TRUE
 120                     WHEN E-INCLUDE
C:/GC-AWORK/customer.cob
run
```

CobGDB opens the program terminal window (the application will run in this separate window.).

```
C:\GC-AWORK\customer.exe
```

go back to the CobGDB screen, and you see a **>** green symbol on the left of statement where initial B (Breakpoint) is present (in our example is at line 103 **B>103**):

```
Prompt dei comandi - q3
CobGDB                    GnuCOBOL GDB Interpreter
 100                10 COLUMN PLUS 2 TO WS-ERROR.
 101
 102            PROCEDURE DIVISION.
B>103           001-START.
 104                SET ENVIRONMENT 'COB_SCREEN_EXCEPTIONS' TO 'Y'
 105                SET ENVIRONMENT 'COB_SCREEN_ESC' TO 'Y'
 106                SET ENVIRONMENT 'ESCDELAY' TO '25'
 107                *>CALL "SYSTEM" USING "chcp 437" WS-STATUS
 108                *>CALL "SYSTEM" USING "mode con: lines=24 cols=80" *> WS-STATU
 109                ACCEPT WS_NUMR FROM LINES
 110                ACCEPT WS-NUMC012 FROM COLUMNS *> WS-STATUS
 111                PERFORM 007-OPEN-FILES
 112                PERFORM UNTIL E-EXIT
 113                    MOVE "MENU" TO WS-OP
 114                    MOVE "CHOOSE AN OPTION" TO WS-STATUS  *> WS-STATUS
 115                    MOVE SPACES TO WS-CHOICE
 116                    DISPLAY SS-CLS
 117                    ACCEPT SS-MENU
 118
 119                    EVALUATE TRUE
 120                        WHEN E-INCLUDE
C:/GC-AWORK/customer.cob
Debugging
```

From that moment on, you can use all the commands (keys) or left-click on corresponding symbols to "animate" and debug the application, example: "**S**" Step (a Step Into), "**N**" Next (a Step Over), "**G**" Go (a Step Out) and so on.

After the **D** command or pressing **D** symbol and during the source code animation, the debugger automatically shows some pop-up windows with variables content from the line in execution.

In order not to slow down the debugging / animation of the program unnecessarily, the automatic display of all variables of the statement in execution is disabled by default at program startup.
To activate the Automatic Display of all variables of the statement in execution use the "**D**" command or click the "**D**" symbol on the Symbol Bar.

2.3. S - Step Command

Proceed with **S** (Step into) command or left click with mouse the ⬇ symbol on the Symbol Bar.

```
Prompt dei comandi - q3
CobGDB                    GnuCOBOL GDB Interpreter
 100                10 COLUMN PLUS 2 TO WS-ERROR.
 101
 102        PROCEDURE DIVISION.
B>103        001-START.
 104            SET ENVIRONMENT 'COB_SCREEN_EXCEPTIONS' TO 'Y'
 105            SET ENVIRONMENT 'COB_SCREEN_ESC' TO 'Y'
 106            SET ENVIRONMENT 'ESCDELAY' TO '25'
 107            *>CALL "SYSTEM" USING "chcp 437" WS-STATUS
 108            *>CALL "SYSTEM" USING "mode con: lines=24 cols=80" *> WS-STATU
 109            ACCEPT WS_NUMR FROM LINES
 110            ACCEPT WS-NUMC012 FROM COLUMNS *> WS-STATUS
 111            PERFORM 007-OPEN-FILES
 112            PERFORM UNTIL E-EXIT
 113                MOVE "MENU" TO WS-OP
 114                MOVE "CHOOSE AN OPTION" TO WS-STATUS  *> WS-STATUS
 115                MOVE SPACES TO WS-CHOICE
 116                DISPLAY SS-CLS
 117                ACCEPT SS-MENU
 118
 119                EVALUATE TRUE
 120                    WHEN E-INCLUDE
C:/GC-AWORK/customer.cob
step
```

the **>** green symbol now is on the following line **>104** :

```
Prompt dei comandi - q3
CobGDB                    GnuCOBOL GDB Interpreter
 100                10 COLUMN PLUS 2 TO WS-ERROR.
 101
 102        PROCEDURE DIVISION.
B 103        001-START.
 >104            SET ENVIRONMENT 'COB_SCREEN_EXCEPTIONS' TO 'Y'
 105            SET ENVIRONMENT 'COB_SCREEN_ESC' TO 'Y'
 106            SET ENVIRONMENT 'ESCDELAY' TO '25'
 107            *>CALL "SYSTEM" USING "chcp 437" WS-STATUS
 108            *>CALL "SYSTEM" USING "mode con: lines=24 cols=80" *> WS-STATU
 109            ACCEPT WS_NUMR FROM LINES
 110            ACCEPT WS-NUMC012 FROM COLUMNS *> WS-STATUS
 111            PERFORM 007-OPEN-FILES
 112            PERFORM UNTIL E-EXIT
 113                MOVE "MENU" TO WS-OP
 114                MOVE "CHOOSE AN OPTION" TO WS-STATUS  *> WS-STATUS
 115                MOVE SPACES TO WS-CHOICE
 116                DISPLAY SS-CLS
 117                ACCEPT SS-MENU
 118
 119                EVALUATE TRUE
 120                    WHEN E-INCLUDE
C:/GC-AWORK/customer.cob
Debugging
```

Now you can proceed with S command or N command or as an example:

- Scroll with cursor down to select line 116 of Procedure Division and type "**B**" (to set a Breakpoint), (you also can simply click with mouse left button on the 116 row number)

- The application displays a "B" on the left of the line 116 (type **B** again - or re-click - when you want to delete the Breakpoint, not do that at this moment)

```
Prompt dei comandi - q3
CobGDB                    GnuCOBOL GDB Interpreter
  100                 10 COLUMN PLUS 2 TO WS-ERROR.
  101
  102         PROCEDURE DIVISION.
B 103         001-START.
 >104             SET ENVIRONMENT 'COB_SCREEN_EXCEPTIONS' TO 'Y'
  105             SET ENVIRONMENT 'COB_SCREEN_ESC' TO 'Y'
  106             SET ENVIRONMENT 'ESCDELAY' TO '25'
  107             *>CALL "SYSTEM" USING "chcp 437" WS-STATUS
  108             *>CALL "SYSTEM" USING "mode con: lines=24 cols=80" *> WS-STATU
  109             ACCEPT WS_NUMR FROM LINES
  110             ACCEPT WS-NUMC012 FROM COLUMNS *> WS-STATUS
  111             PERFORM 007-OPEN-FILES
  112             PERFORM UNTIL E-EXIT
  113                 MOVE "MENU" TO WS-OP
  114                 MOVE "CHOOSE AN OPTION" TO WS-STATUS  *> WS-STATUS
  115                 MOVE SPACES TO WS-CHOICE
B 116                 DISPLAY SS-CLS
  117                 ACCEPT SS-MENU
  118
  119                 EVALUATE TRUE
  120                     WHEN E-INCLUDE
C:/GC-AWORK/customer.cob
breakpoint
```

2.4. G - Go Command

Type **G** (Go) or left click with mouse the ⬆ symbol on the Symbol Bar to execute the program until exit from a PERFORM or a CALL routine (Step Out) or until a stopping point (es. a new Breakpoint) is detected:

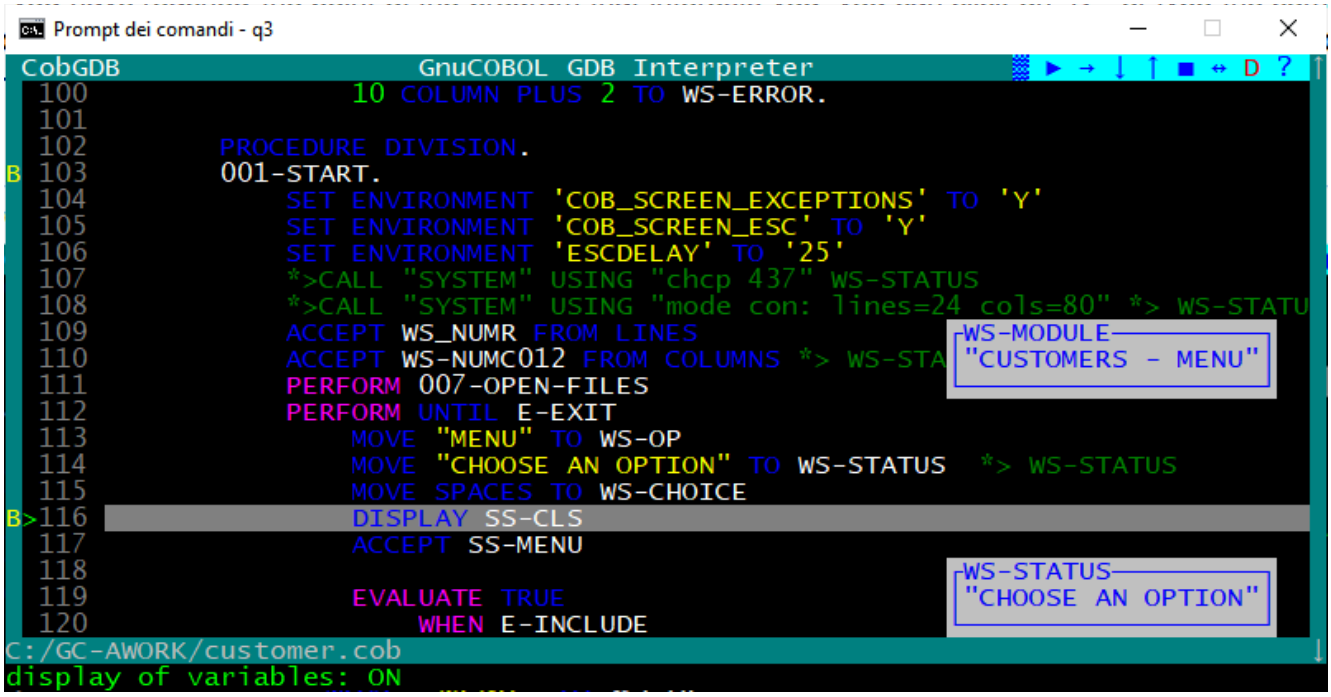
```
Prompt dei comandi - q3
CobGDB                    GnuCOBOL GDB Interpreter
 100                10 COLUMN PLUS 2 TO WS-ERROR.
 101
 102         PROCEDURE DIVISION.
B 103         001-START.
 >104             SET ENVIRONMENT 'COB_SCREEN_EXCEPTIONS' TO 'Y'
 105             SET ENVIRONMENT 'COB_SCREEN_ESC' TO 'Y'
 106             SET ENVIRONMENT 'ESCDELAY' TO '25'
 107             *>CALL "SYSTEM" USING "chcp 437" WS-STATUS
 108             *>CALL "SYSTEM" USING "mode con: lines=24 cols=80" *> WS-STATU
 109             ACCEPT WS_NUMR FROM LINES
 110             ACCEPT WS-NUMC012 FROM COLUMNS *> WS-STATUS
 111             PERFORM 007-OPEN-FILES
 112             PERFORM UNTIL E-EXIT
 113                 MOVE "MENU" TO WS-OP
 114                 MOVE "CHOOSE AN OPTION" TO WS-STATUS  *> WS-STATUS
 115                 MOVE SPACES TO WS-CHOICE
B 116                DISPLAY SS-CLS
 117                 ACCEPT SS-MENU
 118
 119                 EVALUATE TRUE
 120                     WHEN E-INCLUDE
C:/GC-AWORK/customer.cob
go
```

the system reach the second breakpoint at line 116 and displays a green **>** symbol to the left of the line to be executed, see following screen:

```
Prompt dei comandi - q3
CobGDB                    GnuCOBOL GDB Interpreter
 100                10 COLUMN PLUS 2 TO WS-ERROR.
 101
 102         PROCEDURE DIVISION.
B 103         001-START.
 104             SET ENVIRONMENT 'COB_SCREEN_EXCEPTIONS' TO 'Y'
 105             SET ENVIRONMENT 'COB_SCREEN_ESC' TO 'Y'
 106             SET ENVIRONMENT 'ESCDELAY' TO '25'
 107             *>CALL "SYSTEM" USING "chcp 437" WS-STATUS
 108             *>CALL "SYSTEM" USING "mode con: lines=24 cols=80" *> WS-STATU
 109             ACCEPT WS_NUMR FROM LINES
 110             ACCEPT WS-NUMC012 FROM COLUMNS *> WS-STATUS
 111             PERFORM 007-OPEN-FILES
 112             PERFORM UNTIL E-EXIT                    WS-MODULE
 113                 MOVE "MENU" TO WS-OP                "CUSTOMERS - MENU"
 114                 MOVE "CHOOSE AN OPTION" TO WS-STATUS
 115                 MOVE SPACES TO WS-CHOICE
B>116                DISPLAY SS-CLS
 117                 ACCEPT SS-MENU
 118                                                     WS-STATUS
 119                 EVALUATE TRUE                       "CHOOSE AN OPTION"
 120                     WHEN E-INCLUDE
C:/GC-AWORK/customer.cob
Debugging
```

2.5. D - Display Variables Command

The debugger / animator can do an automatic display of all variables content from the statement executed and from the previous statement.
This can slow down the animation / execution and it may not be useful to have it activated everywhere.
Type **D** (Display Variables) command or left click with mouse the **D** symbol on the Symbol bar to enable or disable this display at your convenience.
By moving the mouse over the "**D**" symbol in the Symbol bar you will see a message with the current status of this option displayed at the bottom left of screen: Display of Variables: ON or Display of Variables: OFF.
At the beginning of the program this option is always set to OFF to speed up the animation.
When you have reached the point of the program that interests you, you can click on "**D**" symbol, or type the command "**D**", to activate the automatic display of all the variables of the statement in execution (black frame pop-up windows) and of the previous statement (blue frame pop-up windows).



2.6. H - Show Command

typing the '**H**' command (key) allows you to view the variables on the cursor selected (highlighted) line.

```
Prompt dei comandi - q3
CobGDB                    GnuCOBOL GDB Interpreter
 100                10 COLUMN PLUS 2 TO WS-ERROR.
 101
 102        PROCEDURE DIVISION.
B 103        001-START.
 104            SET ENVIRONMENT 'COB_SCREEN_EXCEPTIONS' TO 'Y'
 105            SET ENVIRONMENT 'COB_SCREEN_ESC' TO 'Y'
 106            SET ENVIRONMENT 'ESCDELAY' TO '25'                 WS-CHOICE
 107     Show Line Variables                                          ""
 108     -SS-CLS: "   CUSTOMERS - MENU                CHOOSE AN OPTION   TU
 109      -SS-CLS: "   CUSTOMERS - MENU                CHOOSE AN OPTIO
 110      -Implicit FILLER: ""
 111      -Implicit FILLER: ""
 112      -Implicit FILLER: ""
 113      -Implicit FILLER: "CUSTOMERS - MENU                 "
 114      -SS-STATUS: "CHOOSE AN OPTION                "
 115      -Implicit FILLER: "CHOOSE AN OPTION                 "
B 116
>117                ACCEPT SS-MENU
 118
 119                EVALUATE TRUE
 120                    WHEN E-INCLUDE
C:/GC-AWORK/customer.cob
E - Edit variable
```

Display the content of variables also clicking right mouse button on a source line, example click right mouse button on line 114 will execute the H command on that line and give you:

```
Prompt dei comandi - q3
CobGDB                    GnuCOBOL GDB Interpreter
 100                10 COLUMN PLUS 2 TO WS-ERROR.
 101
 102        PROCEDURE DIVISION.
B 103        001-START.
 104            SET ENVIRONMENT 'COB_SCREEN_EXCEPTIONS' TO 'Y'
 105            SET ENVIRONMENT 'COB_SCREEN_ESC' TO 'Y'
 106            SET ENVIRONMENT 'ESCDELAY' TO '25'                 WS-CHOICE
 107     Show Line Variables                                          ""
 108     -WS-STATUS: "CHOOSE AN OPTION                "                TU
 109      -WS-STATUS: "CHOOSE AN OPTION                "
 110
 111            PERFORM 007-OPEN-FILES
 112            PERFORM UNTIL E-EXIT
 113                MOVE "MENU" TO WS-OP
 114                MOVE "CHOOSE AN OPTION" TO WS-STATUS   *> WS-STATUS
 115                MOVE SPACES TO WS-CHOICE
B 116                DISPLAY SS-CLS
>117                ACCEPT SS-MENU
 118
 119                EVALUATE TRUE
 120                    WHEN E-INCLUDE
C:/GC-AWORK/customer.cob
E - Edit variable
```

Now at lower left line on screen is a message: E = Edit (change) the variable value.

2.6.1. E - Edit subCommand

Scroll with cursor key UP and DOWN (or with mouse wheel). to select one of the variable.
The key **E** (or clicking again right mouse button) can be used to edit the content of the variable.
Change the value and type Enter to confirm changes or use ESC to exit without changes :

```
Prompt dei comandi - q3
CobGDB                    GnuCOBOL GDB Interpreter
 100                10 COLUMN PLUS 2 TO WS-ERROR.
 101
 102        PROCEDURE DIVISION.
B 103        001-START.
 104            SET ENVIRONMENT 'COB_SCREEN_EXCEPTIONS' TO 'Y'
 105            SET ENVIRONMENT 'COB_SCREEN_ESC' TO 'Y'
 106            SET ENVIRONMENT 'ESCDELAY' TO '25'                 WS-CHOICE
 107     Edit Variable                                                  ""
 108     WS-STATUS: xxxxSE AN OPTION                                    TU
 109
 110
 111            PERFORM 007-OPEN-FILES
 112            PERFORM UNTIL E-EXIT
 113                MOVE "MENU" TO WS-OP
 114                MOVE "CHOOSE AN OPTION" TO WS-STATUS   *> WS-STATUS
 115                MOVE SPACES TO WS-CHOICE
B 116                DISPLAY SS-CLS
 >117                ACCEPT SS-MENU
 118
 119                EVALUATE TRUE
 120                    WHEN E-INCLUDE
C:/GC-AWORK/customer.cob
```

Resulting:

```
Prompt dei comandi - q3
CobGDB                    GnuCOBOL GDB Interpreter
 100                10 COLUMN PLUS 2 TO WS-ERROR.
 101
 102        PROCEDURE DIVISION.
B 103        001-START.
 104            SET ENVIRONMENT 'COB_SCREEN_EXCEPTIONS' TO 'Y'
 105            SET ENVIRONMENT 'COB_SCREEN_ESC' TO 'Y'
 106            SET ENVIRONMENT 'ESCDELAY' TO '25'                 WS-CHOICE
 107     Show Line Variables                                            ""
 108     -WS-STATUS: "xxxxSE AN OPTION                 "                TU
 109      -WS-STATUS: "xxxxSE AN OPTION                 "
 110
 111            PERFORM 007-OPEN-FILES
 112            PERFORM UNTIL E-EXIT
 113                MOVE "MENU" TO WS-OP
 114                MOVE "CHOOSE AN OPTION" TO WS-STATUS   *> WS-STATUS
 115                MOVE SPACES TO WS-CHOICE
B 116                DISPLAY SS-CLS
 >117                ACCEPT SS-MENU
 118
 119                EVALUATE TRUE
 120                    WHEN E-INCLUDE
C:/GC-AWORK/customer.cob
E - Edit variable
```

2.7. V - Variables Command

Type the **V** command (key) to display the list of all program variables:

```
Prompt dei comandi - q3
COBGDB - (R)return (ENTER)expand/contract (E)edit var
 RETURN-CODE: 0
+FILE1 Record: "\000\000\000\000\000\000\000\000\000\000\000\000\000\000\000\0
+WS-MODULE: "CUSTOMERS - MENU                    "
+WS-CHOICE: " "
+FS-STAT: "00"
+WS-ERROR: " "
+WS_NUMR: "025"
+WS-NUMC012: "080"
+BACK-COLOR: "1"
+FRONT-COLOR: "6"
+WS-STATUS: "xxxxSE AN OPTION                "
+WS-ERRMSG: "
+SS-CLS: "                                                                  "
+SS-MENU: "1 - INCLUDE2 - CONSULT3 - UPDATE4 - DELETEX - EXITCHOICE:  "
+SS-RECORD-SCREEN: "  PHONE:              NAME:
+SS-ERROR: "
+COB-CRT-STATUS: "0000"

Debugging
```

2.7.1. Enter - subCommand

Scroll with cursor key UP and DOWN (or with mouse wheel) in this list to the variable WS-MODULE and type Enter: the application opens and displays its subfields WS-MODULE and WS-OP

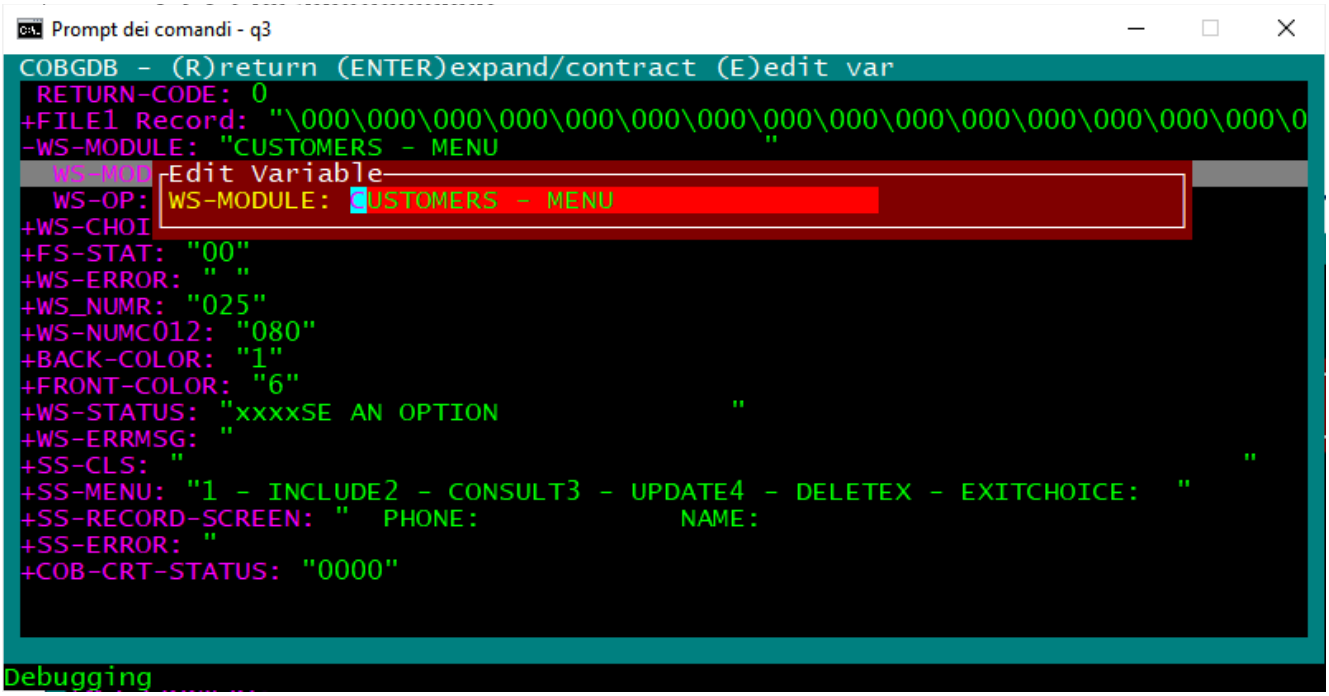
```
Prompt dei comandi - q3
COBGDB - (R)return (ENTER)expand/contract (E)edit var
 RETURN-CODE: 0
+FILE1 Record: "\000\000\000\000\000\000\000\000\000\000\000\000\000\000\000\0
-WS-MODULE: "CUSTOMERS - MENU                    "
  WS-MODULE: "CUSTOMERS - MENU                    "
  WS-OP: "MENU                    "
+WS-CHOICE: " "
+FS-STAT: "00"
+WS-ERROR: " "
+WS_NUMR: "025"
+WS-NUMC012: "080"
+BACK-COLOR: "1"
+FRONT-COLOR: "6"
+WS-STATUS: "xxxxSE AN OPTION                    "
+WS-ERRMSG: "                                                                    "
+SS-CLS: "
+SS-MENU: "1 - INCLUDE2 - CONSULT3 - UPDATE4 - DELETEX - EXITCHOICE:  "
+SS-RECORD-SCREEN: "  PHONE:              NAME:
+SS-ERROR: "
+COB-CRT-STATUS: "0000"

Debugging
```

select WS-MODULE subfield

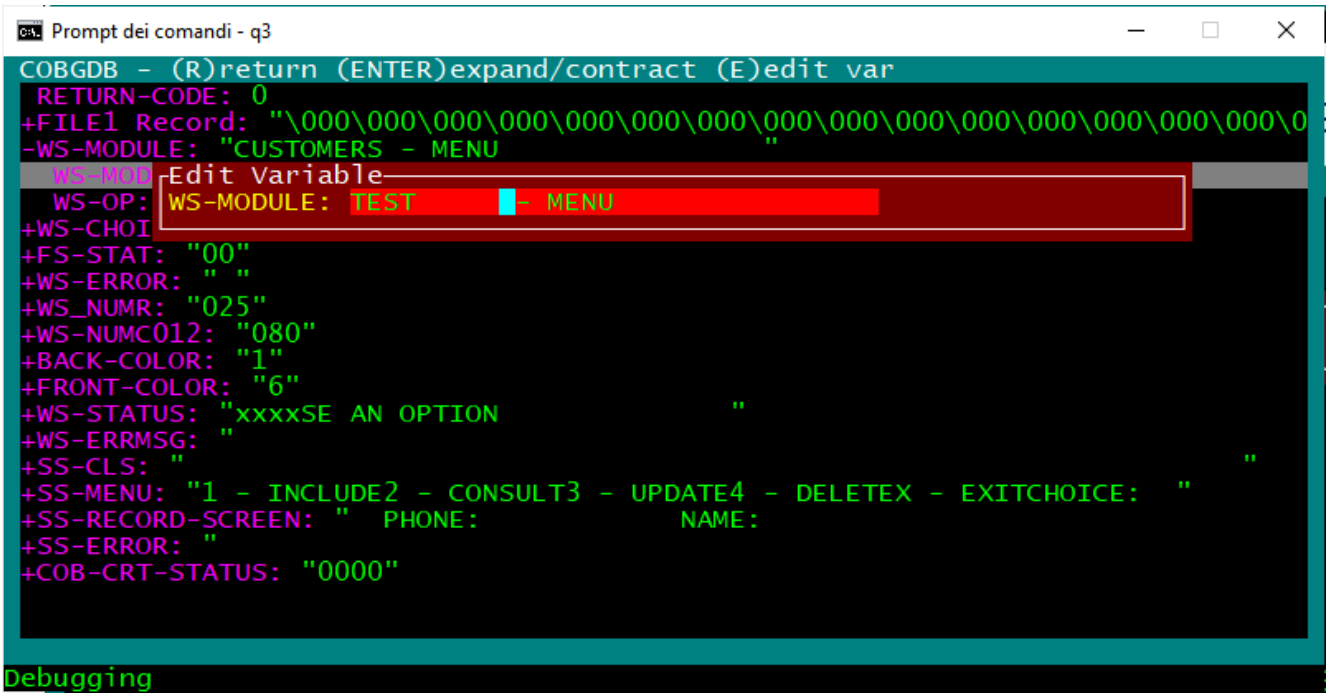
```
Prompt dei comandi - q3
COBGDB - (R)return (ENTER)expand/contract (E)edit var
 RETURN-CODE: 0
+FILE1 Record: "\000\000\000\000\000\000\000\000\000\000\000\000\000\000\000\0
-WS-MODULE: "CUSTOMERS - MENU                    "
  WS-MODULE: "CUSTOMERS - MENU                    "
  WS-OP: "MENU                    "
+WS-CHOICE: " "
+FS-STAT: "00"
+WS-ERROR: " "
+WS_NUMR: "025"
+WS-NUMC012: "080"
+BACK-COLOR: "1"
+FRONT-COLOR: "6"
+WS-STATUS: "xxxxSE AN OPTION                    "
+WS-ERRMSG: "                                                                    "
+SS-CLS: "
+SS-MENU: "1 - INCLUDE2 - CONSULT3 - UPDATE4 - DELETEX - EXITCHOICE:  "
+SS-RECORD-SCREEN: "  PHONE:              NAME:
+SS-ERROR: "
+COB-CRT-STATUS: "0000"

Debugging
```

2.7.2. E - Edit subCommand

Now you can select WS-MODULE subfield whit cursor DOWN or mouse wheel and type "**E**" (Edit)
CobGDB shows a Edit Variable window:



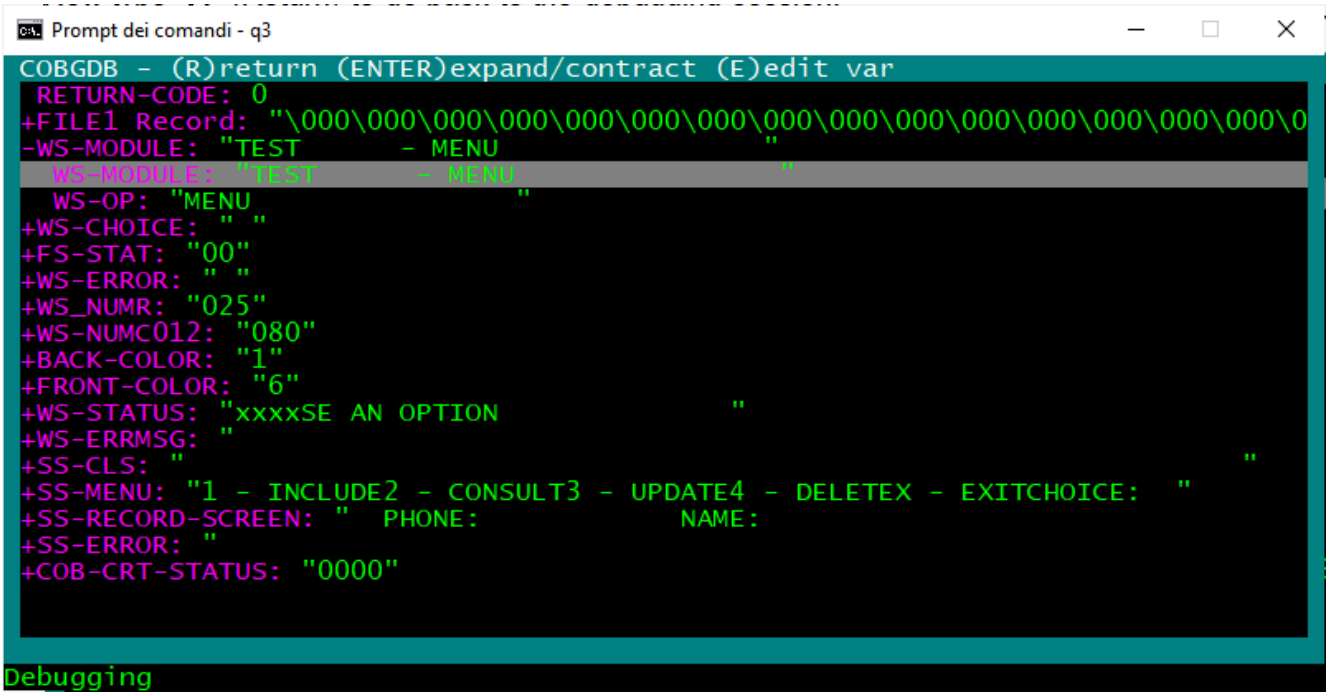
Change "CUSTOMERS" to "TEST" and type Enter to change the value (use ESC to exit without changes) :



2.7.3. R - Return subCommand

WS-Module has new value.

Now type "**R**" (Return) to go back to the debugging session:



2.8. S - Step Command

now you are back in the main debugging window: type **S** (Step into) command or left click with mouse the ⬇ symbol to execute the DISPLAY statement at line 116

```
Prompt dei comandi - q3
COBGDB                    GnuCOBOL GDB Interpreter
 100                10 COLUMN PLUS 2 TO WS-ERROR.
 101
 102          PROCEDURE DIVISION.
B 103          001-START.
 104              SET ENVIRONMENT 'COB_SCREEN_EXCEPTIONS' TO 'Y'
 105              SET ENVIRONMENT 'COB_SCREEN_ESC' TO 'Y'
 106              SET ENVIRONMENT 'ESCDELAY' TO '25'
 107              *>CALL "SYSTEM" USING "chcp 437" WS-STATUS
 108              *>CALL "SYSTEM" USING "mode con: lines=24 cols=80" *> WS-STATU
 109              ACCEPT WS_NUMR FROM LINES                 WS-STATUS
 110              ACCEPT WS-NUMC012 FROM COLUMNS *> WS-STA  "xxxxSE AN OPTION"
 111              PERFORM 007-OPEN-FILES
 112              PERFORM UNTIL E-EXIT
 113                  MOVE "MENU" TO WS-OP
 114                  MOVE "CHOOSE AN OPTION" TO WS-STATUS  *> WS-STATUS
 115                  MOVE SPACES TO WS-CHOICE
B>116                 DISPLAY SS-CLS
 117                  ACCEPT SS-MENU
 118                                                        WS-MODULE
 119                  EVALUATE TRUE                         "TEST       - MENU"
 120                      WHEN E-INCLUDE
C:/GC-AWORK/customer.cob
step
```

in the other application window you can see the result of DISPLAY statement

```
C:\GC-AWORK\customer.exe
TEST        - MENU

xxxxSE AN OPTION
```

go back to debugging window and now the ACCEPT statement will be executed with S command

Type **S** (Step into) command or leftclick with mouse the ⬇ symbol again to execute ACCEPT statement at line 117:

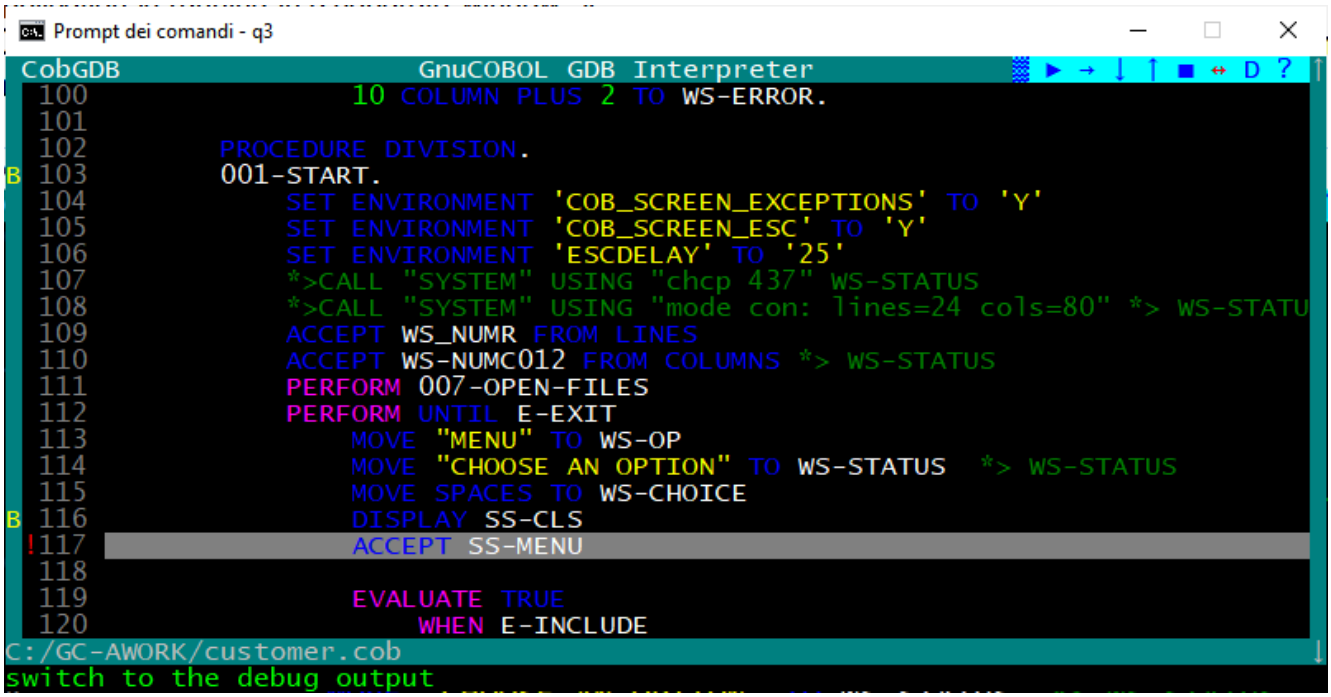
```
Prompt dei comandi - q3
CobGDB                    GnuCOBOL GDB Interpreter
 100                10 COLUMN PLUS 2 TO WS-ERROR.
 101
 102        PROCEDURE DIVISION.
B 103        001-START.
 104            SET ENVIRONMENT 'COB_SCREEN_EXCEPTIONS' TO 'Y'
 105            SET ENVIRONMENT 'COB_SCREEN_ESC' TO 'Y'
 106            SET ENVIRONMENT 'ESCDELAY' TO '25'
 107            *>CALL "SYSTEM" USING "chcp 437" WS-STATUS
 108            *>CALL "SYSTEM" USING "mode con: lines=24 cols=
 109            ACCEPT WS_NUMR FROM LINES
 110            ACCEPT WS-NUMC012 FROM COLUMNS *> WS-STATUS
 111            PERFORM 007-OPEN-FILES
 112            PERFORM UNTIL E-EXIT
 113                MOVE "MENU" TO WS-OP
 114                MOVE "CHOOSE AN OPTION" TO WS-STATUS  *> WS-STATUS
 115                MOVE SPACES TO WS-CHOICE
B 116               DISPLAY SS-CLS
 >117               ACCEPT SS-MENU
 118
 119                EVALUATE TRUE
 120                    WHEN E-INCLUDE
C:/GC-AWORK/customer.cob
step
```

A red **!** exclamation mark appears on the line !117 .
This means that the application is running and a user action is required on application window.

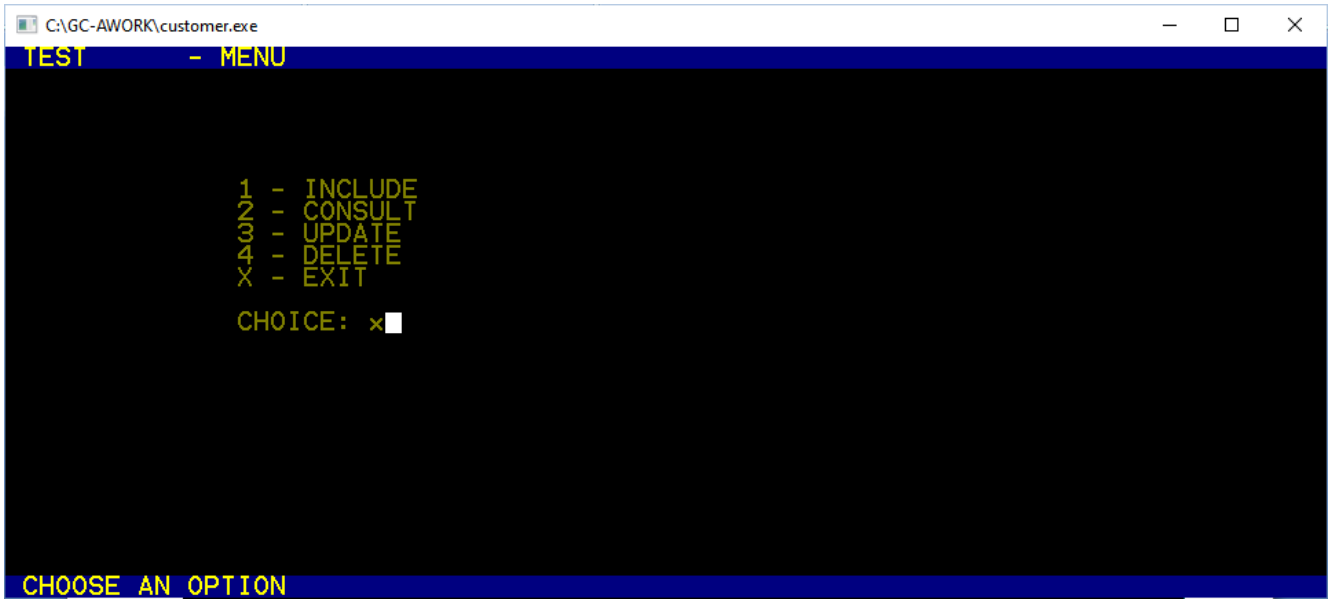
```
Prompt dei comandi - q3
CobGDB                    GnuCOBOL GDB Interpreter
 100                10 COLUMN PLUS 2 TO WS-ERROR.
 101
 102        PROCEDURE DIVISION.
B 103        001-START.
 104            SET ENVIRONMENT 'COB_SCREEN_EXCEPTIONS' TO 'Y'
 105            SET ENVIRONMENT 'COB_SCREEN_ESC' TO 'Y'
 106            SET ENVIRONMENT 'ESCDELAY' TO '25'
 107            *>CALL "SYSTEM" USING "chcp 437" WS-STATUS
 108            *>CALL "SYSTEM" USING "mode con: lines=24 cols=80" *> WS-STATU
 109            ACCEPT WS_NUMR FROM LINES
 110            ACCEPT WS-NUMC012 FROM COLUMNS *> WS-STATUS
 111            PERFORM 007-OPEN-FILES
 112            PERFORM UNTIL E-EXIT
 113                MOVE "MENU" TO WS-OP
 114                MOVE "CHOOSE AN OPTION" TO WS-STATUS  *> WS-STATUS
 115                MOVE SPACES TO WS-CHOICE
B 116               DISPLAY SS-CLS
 !117               ACCEPT SS-MENU
 118
 119                EVALUATE TRUE
 120                    WHEN E-INCLUDE
C:/GC-AWORK/customer.cob
page up
```

2.9. O - Focus Command

The application is running in a separate window.
The '**O** = Focus' command switches the focus from debugging / animation window to the application window.
Instead of using the "**O**" command you can left-click on the ↔ symbol on the Symbol Bar.



The focus now is on the application window where a user action is needed.
In our sample, we type the "X" choice and Enter:



Now the system automatic switch back to the debugger / animation window to continue debugging.
You can see that the ACCEPT statement has been executed, now you are on line >119 :

```
Prompt dei comandi - q3
CobGDB                    GnuCOBOL GDB Interpreter
  101
  102         PROCEDURE DIVISION.
B 103         001-START.
  104             SET ENVIRONMENT 'COB_SCREEN_EXCEPTIONS' TO 'Y'
  105             SET ENVIRONMENT 'COB_SCREEN_ESC' TO 'Y'
  106             SET ENVIRONMENT 'ESCDELAY' TO '25'
  107             *>CALL "SYSTEM" USING "chcp 437" WS-STATUS        WS-CHOICE
  108             *>CALL "SYSTEM" USING "mode con: lines=24 cols=          "x" TU
  109             ACCEPT WS_NUMR FROM LINES
  110             ACCEPT WS-NUMC012 FROM COLUMNS *> WS-STATUS
  111             PERFORM 007-OPEN-FILES
  112             PERFORM UNTIL E-EXIT
  113                 MOVE "MENU" TO WS-OP
  114                 MOVE "CHOOSE AN OPTION" TO WS-STATUS  *> WS-STATUS
  115                 MOVE SPACES TO WS-CHOICE
B 116                 DISPLAY SS-CLS
  117                 ACCEPT SS-MENU
  118
>119                  EVALUATE TRUE
  120                     WHEN E-INCLUDE
  121                         PERFORM 002-INCLUDE THRU 002-INCLUDE-END
C:/GC-AWORK/customer.cob
Debugging
```

2.10. Pop-up Variable windows

During a debugging session CobGDB shows variable content.
Blue frame and values ..... : variables of executing cobol statement
Black frame and values ..... : variables of last executed cobol statement.

Sample:

```
Prompt dei comandi - q2
COBGDB                    GnuCOBOL GDB Interpreter
 262   continue after 0.5 seconds                                  wColor
 263                                                                   1
 264   continue.
 265 DisplayCoverEx. exit.                                         wRow
 266                                                                "03"
 267 TetraminoDisplay.
 268   perform varying wRow from 1 by 1 until wRow > 4
 269     perform varying wCol from 1 by 1 until wCol > 4
 270         compute wLinD = wBaseLin + wRow
 271         compute wColD = wBaseCol + wCol
 272         if wBlockEle (wRow wCol) = 'X'
B>273             display wChar at line wLinD column wColD :BCO: blue :FCO: wCol
 274         end-if                                                wCol
 275     end-perform                                                  3
 276   end-perform
 277   continue.
 278 TetraminoDisplayEx. exit.
 279
 280 TetraminoSet wColD      wLinD      wChar          wBlock
 281     move spa    41  o   9      "X"          "  X   X   X   X"
 282     evaluate        n
C:/GC-AWORK/GC99TETRIS.COB
go
```

See also the “D” command to enable or disable these pop-up windows.

2.11. F - File Command

To show this command we use following sample:

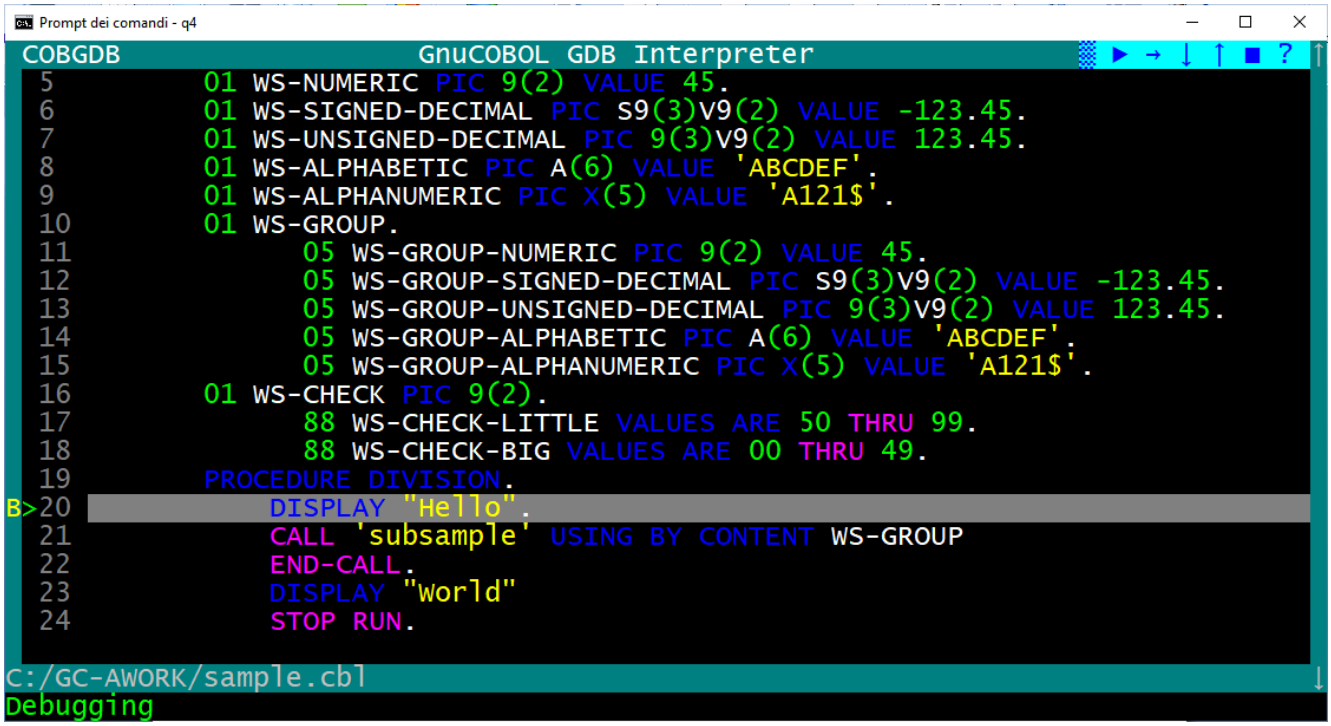
```
cobgdb sample.cbl subsample.cbl subsubsample.cbl -x -lpdcurses
```

where sample.cbl is the main program; it calls
--> subsample.cbl; it calls
--> subsubsample.cbl

Source code is at https://github.com/marcsosduma/cobgdb/tree/main/resources.
This will create a single sample.exe executable.

This example shows that when you need to debug only subsample.cbl or only subsubsample.cbl you need to execute CobGDB with all three programs.

CobGDB sets the B breakpoint at first executable statement of first program "sample.cbl".
here use the R Run command to start the debugging session.



Now you can type the **F File** command and you will have the "Source Files" window.
In this sample we select the second program in the list (subsample.cbl) and type Enter.

```
Prompt dei comandi - q4
COBGDB                    GnuCOBOL GDB Interpreter
 5          01 WS-NUMERIC PIC 9(2) VALUE 45.
 6          01 WS-SIGNED-DECIMAL PIC S9(3)V9(2) VALUE -123.45.
 7          01 WS-UNSIGNED-DECIMAL PIC 9(3)V9(2) VALUE 123.45.
 8          01 WS-ALPHABETIC PIC A(6) VALUE 'ABCDEF'.
 9          01 WS-ALPHANUMERIC PIC X(5) VALUE 'A121$'.
10      Source Files
11      C:/GC-AWORK/sample.cbl
12      C:/GC-AWORK/subsample.cbl                                  .45.
13      C:/GC-AWORK/subsubsample.cbl                               .45.
14
15
16
17
18
19
B>20
21
22              END-CALL.
23              DISPLAY "World"
24              STOP RUN.
C:/GC-AWORK/sample.cbl
Debugging
```

CobGDB shows the selected program source code where in this sample we type a B command at line 14.

```
Prompt dei comandi - q4
COBGDB                    GnuCOBOL GDB Interpreter
 1          IDENTIFICATION DIVISION.
 2          PROGRAM-ID. subsample.
 3          ENVIRONMENT DIVISION.
 4          DATA DIVISION.
 5          WORKING-STORAGE SECTION.
 6          LINKAGE SECTION.
 7          01 WS-GROUP.
 8                05 WS-GROUP-NUMERIC PIC 9(2).
 9                05 WS-GROUP-SIGNED-DECIMAL PIC S9(3)V9(2).
10                05 WS-GROUP-UNSIGNED-DECIMAL PIC 9(3)V9(2).
11                05 WS-GROUP-ALPHABETIC PIC A(6).
12                05 WS-GROUP-ALPHANUMERIC PIC X(5).
13          PROCEDURE DIVISION USING WS-GROUP.
B 14            DISPLAY WS-GROUP-ALPHABETIC.
15              CALL 'subsubsample' USING BY CONTENT WS-GROUP
16              END-CALL.
17          END PROGRAM subsample.
18
C:/GC-AWORK/subsample.cbl
breakpoint
```

now we type the **F** command again, then select the "sample.cbl" program and press Enter

```
Prompt dei comandi - q4
COBGDB                    GnuCOBOL GDB Interpreter
  1         IDENTIFICATION DIVISION.
  2         PROGRAM-ID. subsample.
  3         ENVIRONMENT DIVISION.
  4         DATA DIVISION.
  5         WORKING-STORAGE SECTION.
  6      Source Files
  7      C:/GC-AWORK/subsample.cbl
  8      C:/GC-AWORK/sample.cbl
  9      C:/GC-AWORK/subsubsample.cbl
  10
  11
  12
  13
B 14
  15
  16
  17
  18
C:/GC-AWORK/subsample.cbl
```

now we are back to the sample.cbl program to continue the debugging session as we need.

```
Prompt dei comandi - q4
COBGDB                    GnuCOBOL GDB Interpreter
  5         01 WS-NUMERIC PIC 9(2) VALUE 45.
  6         01 WS-SIGNED-DECIMAL PIC S9(3)V9(2) VALUE -123.45.
  7         01 WS-UNSIGNED-DECIMAL PIC 9(3)V9(2) VALUE 123.45.
  8         01 WS-ALPHABETIC PIC A(6) VALUE 'ABCDEF'.
  9         01 WS-ALPHANUMERIC PIC X(5) VALUE 'A121$'.
  10        01 WS-GROUP.
  11              05 WS-GROUP-NUMERIC PIC 9(2) VALUE 45.
  12              05 WS-GROUP-SIGNED-DECIMAL PIC S9(3)V9(2) VALUE -123.45.
  13              05 WS-GROUP-UNSIGNED-DECIMAL PIC 9(3)V9(2) VALUE 123.45.
  14              05 WS-GROUP-ALPHABETIC PIC A(6) VALUE 'ABCDEF'.
  15              05 WS-GROUP-ALPHANUMERIC PIC X(5) VALUE 'A121$'.
  16        01 WS-CHECK PIC 9(2).
  17              88 WS-CHECK-LITTLE VALUES ARE 50 THRU 99.
  18              88 WS-CHECK-BIG VALUES ARE 00 THRU 49.
  19        PROCEDURE DIVISION.
B>20            DISPLAY "Hello".
  21            CALL 'subsample' USING BY CONTENT WS-GROUP
  22            END-CALL.
  23            DISPLAY "World"
  24            STOP RUN.
C:/GC-AWORK/sample.cbl
Debugging
```

2.12. R - Run Command

If you click the Run command during a debug session you will receive a confirmation request, because Yes will restart a new debugging session from first Procedure Division executable statement:

```
Prompt dei comandi - q3
COBGDB                    GnuCOBOL GDB Interpreter
 100                10 COLUMN PLUS 2 TO WS-ERROR.
 101
 102         PROCEDURE DIVISION.
B 103         001-START.
 104             SET ENVIRONMENT 'COB_SCREEN_EXCEPTIONS' TO 'Y'
 105             SET ENVIRONMENT 'COB_SCREEN_ESC' TO 'Y'
 106             SET ENVIRONMENT 'ESCDELAY' TO '25'
 107             *>CALL "SYSTEM" USING "chcp 437" WS-STATUS
 108       Message                                          > WS-STATU
 109       Would you like to "Run" the program again ? (= Restart)
>110                                Yes    No
 111
 112             PERFORM UNTIL E-EXIT
 113                 MOVE "MENU" TO WS-OP
 114                 MOVE "CHOOSE AN OPTION" TO WS-STATUS   *> WS-STATUS
 115                 MOVE SPACES TO WS-CHOICE                 WS-NUMC012
 116                 DISPLAY SS-CLS                                  0
 117                 ACCEPT SS-MENU
 118                                                            WS_NUMR
 119                 EVALUATE TRUE                                   25
 120                     WHEN E-INCLUDE
C:/GC-AWORK/customer.cob
run
```

Note: if program has DECLARATIVES then the first automatic B Breakpoint will be settled at first executable PROCEDURE DIVISION statement that is the one after END DECLARATIVES, see following sample:

```
Prompt dei comandi - q1
COBGDB                    GnuCOBOL GDB Interpreter
 310            MOVE "ok" TO w-flag
 311         ELSE
 312            MOVE "N " TO w-flag
 313         END-IF
 314     ELSE
 315         DISPLAY "Error " w-fsRep " on REPORT FILE "
 316         STOP RUN
 317     END-IF.
 318 ex-err-Rep-x. EXIT.
 319 END DECLARATIVES.
 320
 321 *> ***********************************************************************
 322 *>
 323 *> ***********************************************************************
B>324 MAIN1 SECTION.
 325 MAIN-LOOP.
 326     perform InitialSettings thru InitialSettingsEx
 327
 328     PERFORM UNTIL WCRT-STATUS = K-ESCAPE
 329         DISPLAY screen-menu
 330         ACCEPT  screen-menu
C:/GC-AWORK/MASTER.cob
Debugging
```

2.13. W - Window Size command

When you start a debugging session, the screen size is 24 rows x 80 columns.

```
Prompt dei comandi - z1
COBGDB                    GnuCOBOL GDB Interpreter
 703        MOVE 1 TO U-CNT
 704
 705        *> EVALUATE WHITE
 706        EVALUATE TRUE
 707            WHEN PLAYER-TURN = 'W'
>708                PERFORM UNTIL INP-X-VAR-N = W-X-VAR(U-CNT)
 709                          AND INP-Y-VAR   = W-Y-POS(U-CNT)
 710                          OR  U-CNT         >= 16
 711
 712                          *> DEBUG                                  INP-Y-VAR
 713                          *> display INP-X-VAR-N     at 2260            "7"
 714                          *> display W-X-VAR(U-CNT) at 2265
 715                          *> display INP-Y-VAR       at          WS-COUNTERS
 716                          *> display W-Y-POS(U-CNT) at          "917040000000000"
 717                          *> DISPLAY U-CNT           at
 718            COORDINATES
 719            "051508WKY041208WQY010308WRY082408WRY020608WHY072108WHY03090
 720
 721                    ADD 1 TO U-CNT                                 INP-X-VAR-N
 722                END-PERFORM                                            "8"
 723                EVALUATE TRUE
C:/GC-AWORK/GC99CHESS.COB
Debugging
```

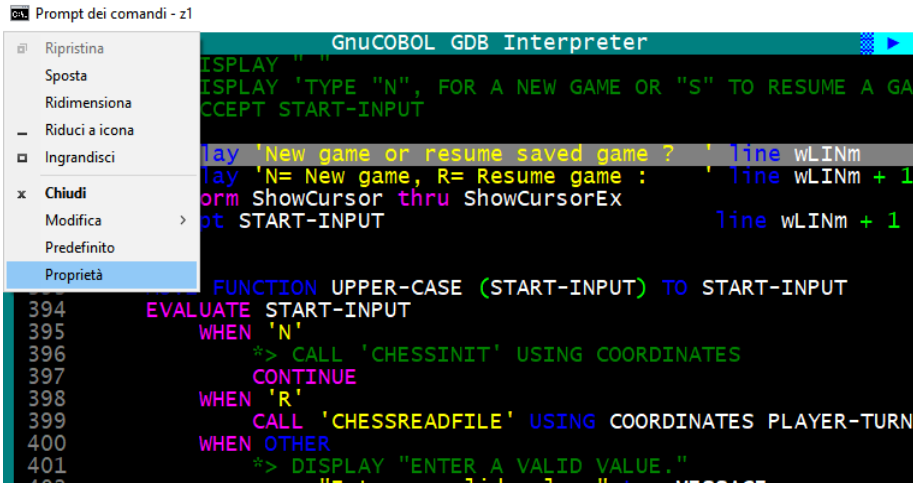
Type **W** (Window Size) command to switch between two window size of the debugger : 24 x 80 or 34 x 132.

```
Prompt dei comandi - z1
COBGDB                    GnuCOBOL GDB Interpreter
 703        MOVE 1 TO U-CNT
 704
 705        *> EVALUATE WHITE
 706        EVALUATE TRUE
 707            WHEN PLAYER-TURN = 'W'
>708                PERFORM UNTIL INP-X-VAR-N = W-X-VAR(U-CNT)
 709                          AND INP-Y-VAR   = W-Y-POS(U-CNT)
 710                          OR  U-CNT         >= 16
 711
 712                          *> DEBUG
 713                          *> display INP-X-VAR-N     at 2260                INP-Y-VAR
 714                          *> display W-X-VAR(U-CNT) at 2265                     "7"
 715                          *> display INP-Y-VAR       at 2360
 716                          *> display W-Y-POS(U-CNT) at 2365   COORDINATES
 717                          *> DISPLAY U-CNT           at 2460  "051508WKY041208WQY010308WRY082408WRY020608WHY072108WHY03090
 718                          *> accept omitted
 719                          *> END DEBUG                                                  INP-X-VAR-N
 720                                                                                            "8"
 721                    ADD 1 TO U-CNT
 722                END-PERFORM                                                          WS-COUNTERS
 723                EVALUATE TRUE                                                     "917040000000000"
 724                    WHEN W-PIECE(U-CNT) = '--'
 725                        SET NOT-VALID TO TRUE                                     WS-COUNTERS.U-CNT
 726                        *> DISPLAY "NO VALID PIECE HERE"                                   4
 727                        move "No valid piece here." to wMESSAGE
 728                        perform DisplayMessage thru DisplayMessageEx
 729                        PERFORM GA-ACCEPT-CORD
 730                    WHEN INP-Y-VAR = W-Y-POS(U-CNT)
 731                        *> DISPLAY "YOU HAVE SELECTED " W-PIECE(U-CNT) " IN POSITION " INP-X-VAR INP-Y-VAR
 732                        string "Selecterd " W-PIECE(U-CNT) " in pos. " INP-X-VAR INP-Y-VAR into wMESSAGE
 733                        perform DisplayMessage thru DisplayMessageEx
C:/GC-AWORK/GC99CHESS.COB
Debugging
```

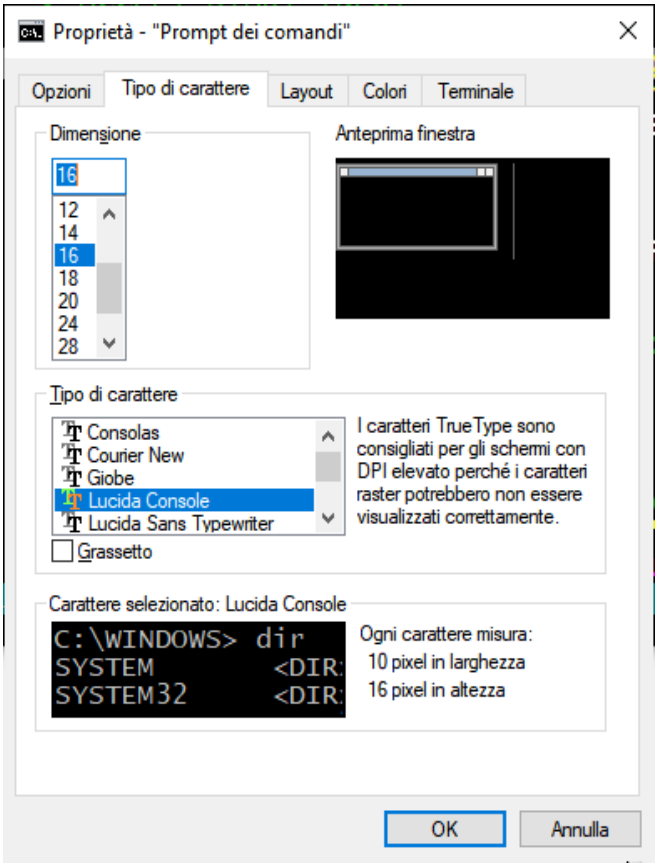
This can be useful when you write GnuCOBOL code with the GnuCOBOL *FREE FORMAT* source option where each line of code can be longer than the classic 80 characters of GnuCOBOL *FIXED FORMAT*.

Warning 1.

Make sure the system font size is not too large otherwise the W command cannot display the 132 columns (your screen will "blink"). For a Windows environment use "Property" menu item to chek or change the Font size:



and then select a suitable font, for example Lucida Console of 16 might be adequate.



Warning 2.
If you are running on Windows 11, it's mandatory to switch the "default terminal" back to "*Windows Console Host"*. The new "*Windows Terminal*", which is the default on Windows 11, does not allow changing the console window size in CobGDB.

The **W command** in CobGDB will not work properly unless this change is made:

1. Open Windows Terminal (using the **cmd** command)
2. Click the down arrow **v** next to the tab bar and select **Settings**
3. In the left-hand menu, go to **Startup**.
4. Under **Default terminal application**, select **Windows Console Host**.
5. Click **Save**.

2.14. Q - Quit Command

To close the debug session use the **Q Quit** command or left click with mouse the ■ symbol on the Symbol Bar.

```
Prompt dei comandi
  102         PROCEDURE DIVISION.
B 103         001-START.
  104             SET ENVIRONMENT 'COB_SCREEN_EXCEPTIONS' TO 'Y'
  105             SET ENVIRONMENT 'COB_SCREEN_ESC' TO 'Y'
  106             SET ENVIRONMENT 'ESCDELAY' TO '25'
  107             *>CALL "SYSTEM" USING "chcp 437" WS-STATUS
  108             *>CALL "SYSTEM" USING "mode con: lines=24 cols=80" *> WS-STATU
  109             ACCEPT WS_NUMR FROM LINES
 >110             ACCEPT WS-NUMC012 FROM COLUMNS *> WS-STATUS
  111             PERFORM 007-OPEN-FILES
  112             PERFORM UNTIL E-EXIT
  113                 MOVE "MENU" TO WS-OP
  114                 MOVE "CHOOSE AN OPTION" TO WS-STATUS  *> WS-STATUS
  115                 MOVE SPACES TO WS-CHOICE                 WS-NUMC012
  116                 DISPLAY SS-CLS                                  0
  117                 ACCEPT SS-MENU
  118
  119                 EVALUATE TRUE
  120                     WHEN E-INCLUDE
C:/GC-AWORK/customer.cob
quit
The end of the COBGDB execution.

C:\GC-AWORK>
```

2.15. A - Attach Command

You may debug your COBOL program attaching to a running Local process or running Remote Debugger (GDBServer).

Prerequiste: your programs are all compiled for debug (use the -g, a cobc compiler option), because you just compile everything for debug because that's no big overhead and you can attach and use system corefiles at any time. CobGDB needs the .cob, .c, .c.h, and .c.l.h files in order to perform debugging.

- Scenario 1. Start a program from the command line.
You are "somewhere in the application" and find the current state to be strange or can reproduce the bug by something you do. Now you just open the debugger from a different terminal, **Attach** to the program, debug normally, then either kill or detach.

- Scenario 2. The program has no terminal at all (like with [web]services, Java/C/HTML frontends, runs in the background, is started by other COBOL programs, ...) it is just "somehow started". Now you just open the debugger from a different terminal, **Attach** to the program, debug normally, then either kill or detach.

- Scenario 3. You can run GDB/GDBSERVER remotely using the **Attach** command. CobGDB will prompt you to provide the server and port in the format server:port or the PID of the application.
Example:

- **`server:port = localhost:5555`**
- **`PID = 9112`**

Command line:

- **`cobgdb --connect localhost:5555 prog.cob`**

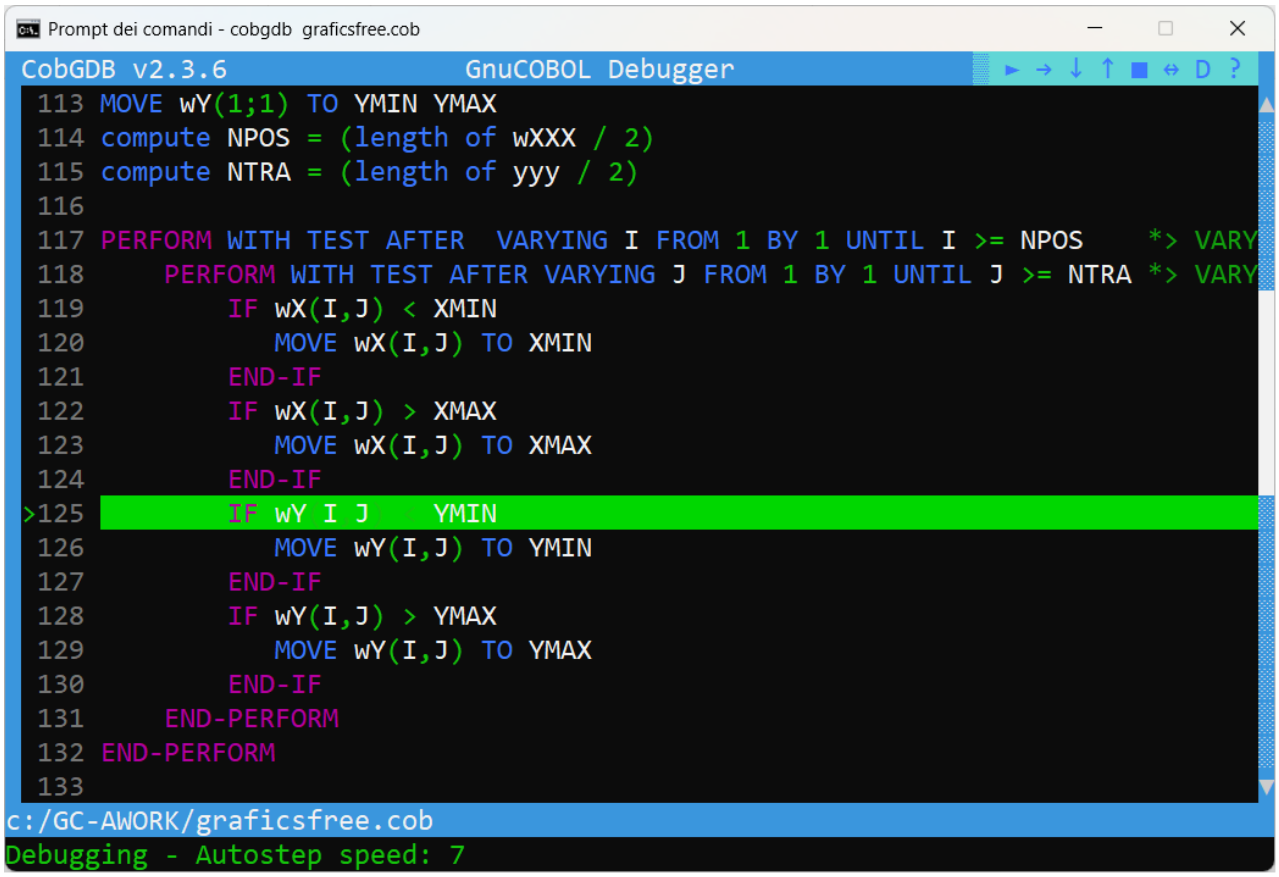
Just to note how debug CGI programs: Start **`gdbserver cgiexecutable`** and just get its PID after starting (possibly have it waiting inside a loop) and ensuring the setup provides rights to ptrace (debug) that process, then use the following cobgdb with its "A"ttach option to attach to that.

```
Prompt dei comandi - q1
CobGDB                    GnuCOBOL GDB Interpreter
 327 *> ***********************************************************************
 328 *>
 329 *> ***********************************************************************
B 330 MAIN1 SECTION.
 >331 MAIN-LOOP.
 332   perform InitialSettings thru InitialSettingsEx
 333
 334   PERFORM UNTIL wCRT-STATUS = K-ESCAPE
 335       DISPLAY screen-menu
 336       call 'TIMEDISP'
 337     Attach - server:port or pid
 338     Enter:
 339
 340           END-ACCEPT
 341       end-perform
 342
 343       IF wCRT-STATUS = K-ESCAPE then
 344          EXIT perform
 345       END-IF
 346       IF w-char2 NOT < 01 AND w-char2 NOT > 10 THEN
 347          MOVE w-char2 TO w-Key
C:/GC-AWORK/MASTER.cob
Connecting
```

3. Other Commands

3.1. CTRL-A - Autostep command

Press CTRL-A to animate the execution.
CobGDB will perform repeatedly the S - Step command until it reaches the next Breakpoint.
A message at the bottom left of the screen shows that Autostep is active and its speed.
Press ESC to stop the Autostep execution.

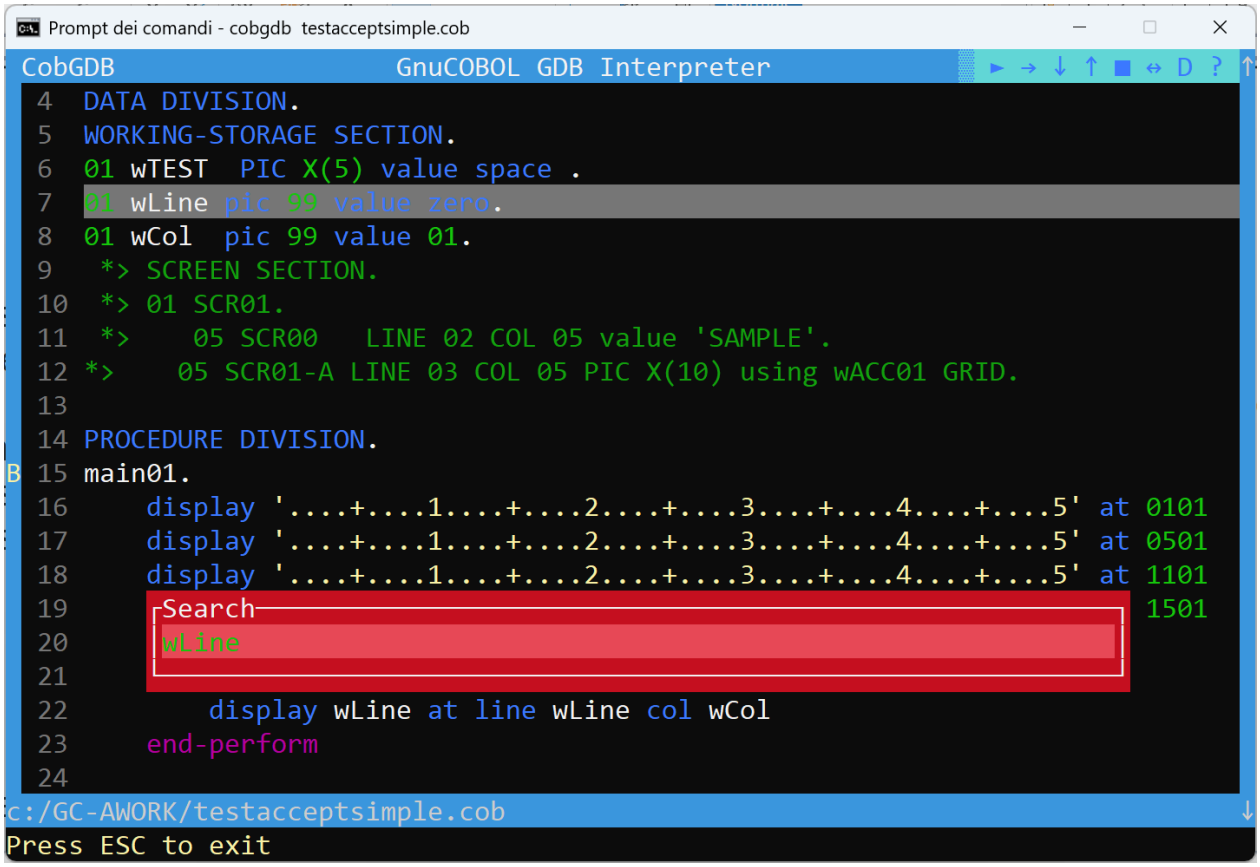


During Autostep, use “+” or "-" keys to increase or decrease the Autostep speed (speed range is from 1 to 10).
The selected Autostep Speed will be automatically saved at the end of the debugging session in the second record of a dedicated file **`C:\USERS\username\cobgbd.cfg`**.
In subsequent debugging sessions, this saved Autostep Speed will be used when the user press CTRL-A.

When Autostep is running, you can use some other commands. For example, the W command to change the screen size, the D command to toggle the display of variable values, and so on.

3.2. CTRL-F - Find command

Press CTRL-F to display the “Search pop-up”. Type the text and press Enter to start the search foreword.
The system performs the search and keeps the pop-up on the screen with the text already filled in, whether the string was found or not. Press Enter again to repeat the search for the same string.
Press ESC when you want to exit the search.



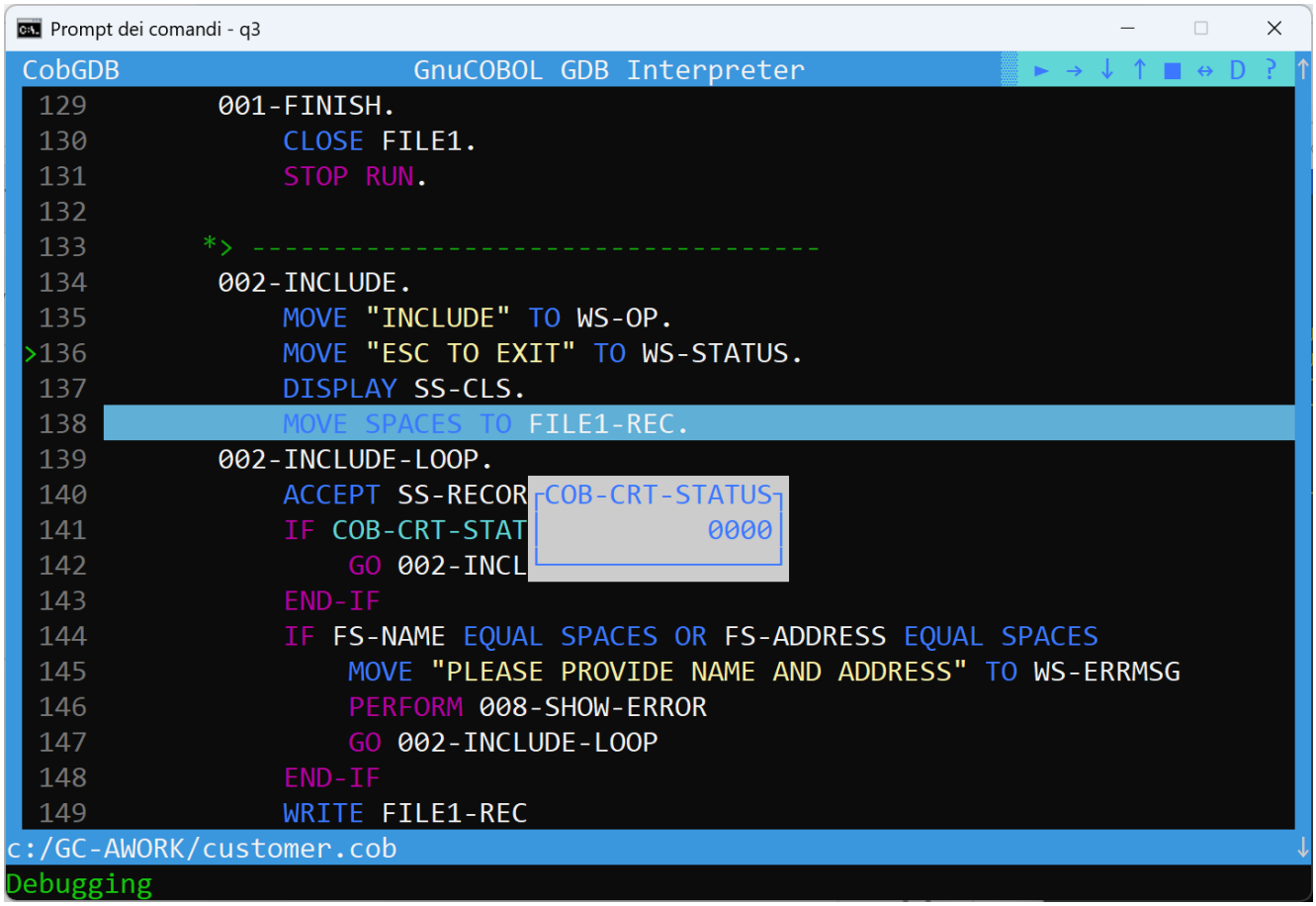
3.3. Change the Current Line background color

Use the + or – keys to change the background color of the "current line" to have a better visibility.
There are 256 available background colors. See also at:
https://jakob-bagterp.github.io/colorist-for-python/ansi-escape-codes/extended-256-colors/#sequence-parts

This new background color will be automatically saved at the end of the debugging session in the first record of a dedicated file **`C:\USERS\username\cobgbd.cfg`**
In subsequent debugging sessions, this color will be used for the background of the "current line".
To restore the default background color simply delete o rename this file.

3.4. Display Variable content by hover the mouse

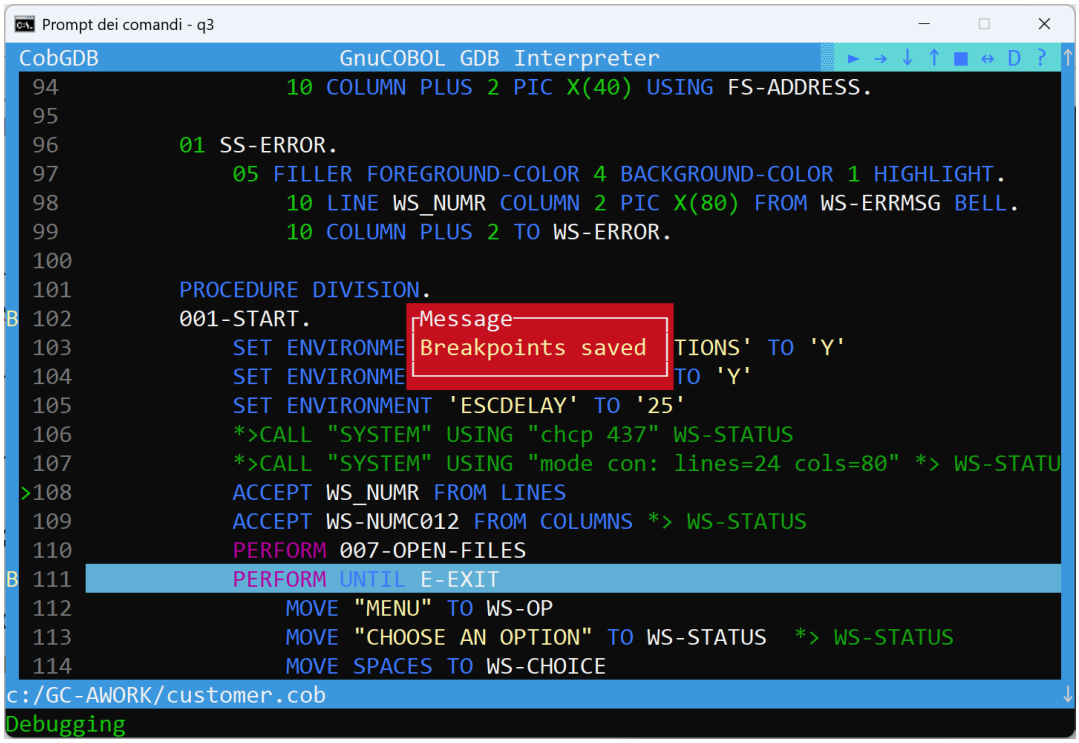
Hover your mouse over the name of a variable (either in Procedure Division or Data Division) and hold it for 2 seconds. A pop-up will appear with the contents of that variable. Move the mouse and the pop-up disappear



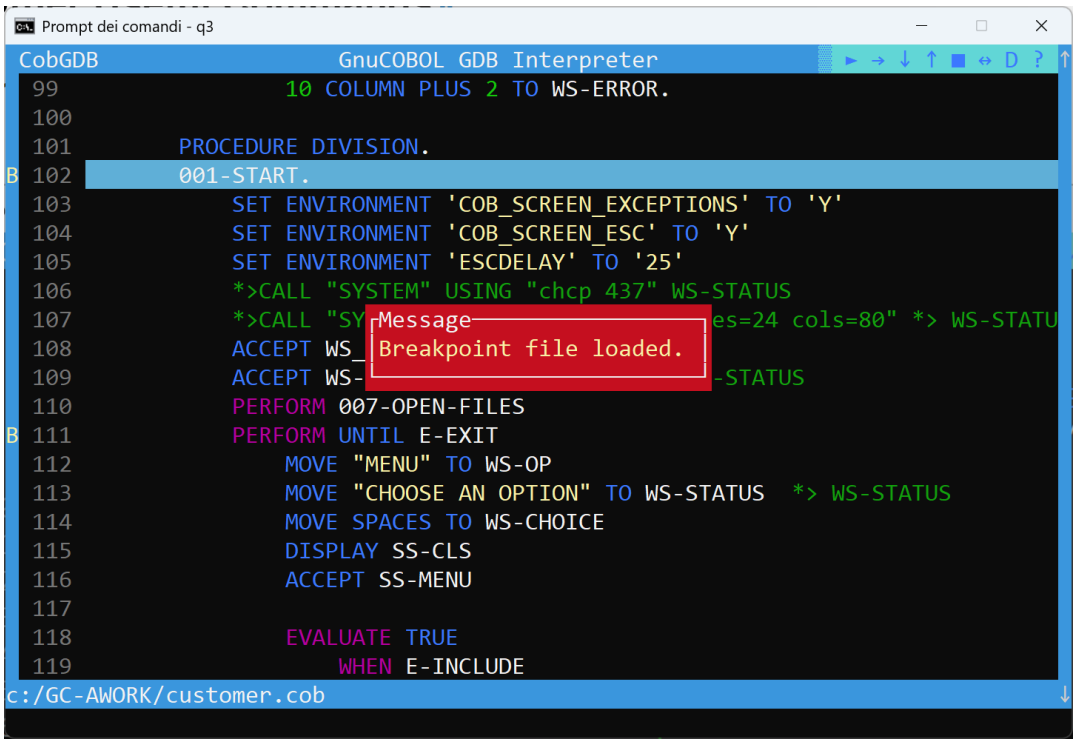
3.5. CTRL-S and CTRL-B - Save and Load Breakpoints

Type **CTRL-S** to save Breakpoints from current session.
CobGDB will create the file **`program-name.bkp`** es. **`customer.bkp`** with following sample records content:

```
c:/GC-AWORK/customer.cob 102
c:/GC-AWORK/customer.cob 111
```



Type **CTRL-B** to load Breakpoints saved from the previous debugging session:



3.6. Debugging a pre-compiled Program

You can use CobGDB to debug a previously generated executable file ex. **`prog.exe`**.
To do this, you must first compile the GnuCOBOL program **`prog.cob`** with these options:

```
cobc -g -fsource-location -ftraceall -v -O0 -x prog.cob prog2.cob ...
```

To start debugging without recompile the program, run cobgdb using the **`--exe`** directive as follows:

Windows: **`cobgdb --exe prog.exe`**

Linux: **`cobgdb --exe prog`**

3.7. Debugging sub programs

We use following sample programs **`p0.cob`** and **`p1.cob`** to show this feature:

```
>>SOURCE FORMAT IS FREE
IDENTIFICATION DIVISION.
PROGRAM-ID. p0.
DATA DIVISION.
WORKING-STORAGE SECTION.
01  WS-NUMERIC                     PIC 9(2)       VALUE 45.
01  WS-SIGNED-DECIMAL              PIC S9(3)V9(2) VALUE -123.45.
01  WS-UNSIGNED-DECIMAL            PIC 9(3)V9(2)  VALUE 123.45.
01  WS-ALPHABETIC                  PIC A(6)       VALUE 'ABCDEF'.
01  WS-ALPHANUMERIC                PIC X(5)       VALUE 'A121$'.
01  WS-GROUP.
    05  WS-GROUP-NUMERIC           PIC  9(2)      VALUE 45.
    05  WS-GROUP-SIGNED-DECIMAL    PIC S9(3)V9(2) VALUE -123.45.
    05  WS-GROUP-UNSIGNED-DECIMAL  PIC  9(3)V9(2) VALUE 123.45.
    05  WS-GROUP-ALPHABETIC        PIC  A(6)      VALUE 'ABCDEF'.
    05  WS-GROUP-ALPHANUMERIC      PIC  X(5)      VALUE 'A121$'.
    05  WS-MSG                     PIC  X(70).
01  WS-CHECK PIC 9(2).
    88 WS-CHECK-LITTLE VALUES ARE 50 THRU 99.
    88 WS-CHECK-BIG    VALUES ARE 00 THRU 49.

PROCEDURE DIVISION.
    DISPLAY "PROGRAM P0"
    DISPLAY WS-GROUP-NUMERIC
    DISPLAY WS-GROUP-SIGNED-DECIMAL
    DISPLAY WS-GROUP-UNSIGNED-DECIMAL
    DISPLAY WS-GROUP-ALPHABETIC
    DISPLAY WS-GROUP-ALPHANUMERIC
    DISPLAY WS-MSG

    MOVE 'Program "P1" was called by "P0"' TO WS-MSG
    CALL 'p1' USING BY CONTENT WS-GROUP END-CALL

    DISPLAY 'BACK TO "P0", received the message:'
    DISPLAY WS-GROUP-NUMERIC
    DISPLAY WS-GROUP-SIGNED-DECIMAL
    DISPLAY WS-GROUP-UNSIGNED-DECIMAL
    DISPLAY WS-GROUP-ALPHABETIC
    DISPLAY WS-GROUP-ALPHANUMERIC
    DISPLAY WS-MSG

   STOP RUN.
```

```
>>SOURCE FORMAT IS FREE
IDENTIFICATION DIVISION.
PROGRAM-ID. p1.
ENVIRONMENT DIVISION.
DATA DIVISION.
WORKING-STORAGE SECTION.
LINKAGE SECTION.
01 WS-GROUP.
    05 WS-GROUP-NUMERIC           PIC 9(2).
    05 WS-GROUP-SIGNED-DECIMAL    PIC S9(3)V9(2).
    05 WS-GROUP-UNSIGNED-DECIMAL  PIC 9(3)V9(2).
    05 WS-GROUP-ALPHABETIC        PIC A(6).
    05 WS-GROUP-ALPHANUMERIC      PIC X(5).
    05 WS-MSG                     PIC X(70).

PROCEDURE DIVISION USING WS-GROUP.
    DISPLAY "PROGRAM P1"
    DISPLAY "Message received: " WS-MSG
    DISPLAY 'BACK IN "P1" receiving the message:'
    DISPLAY WS-MSG
    GOBACK.
END PROGRAM p1.
```

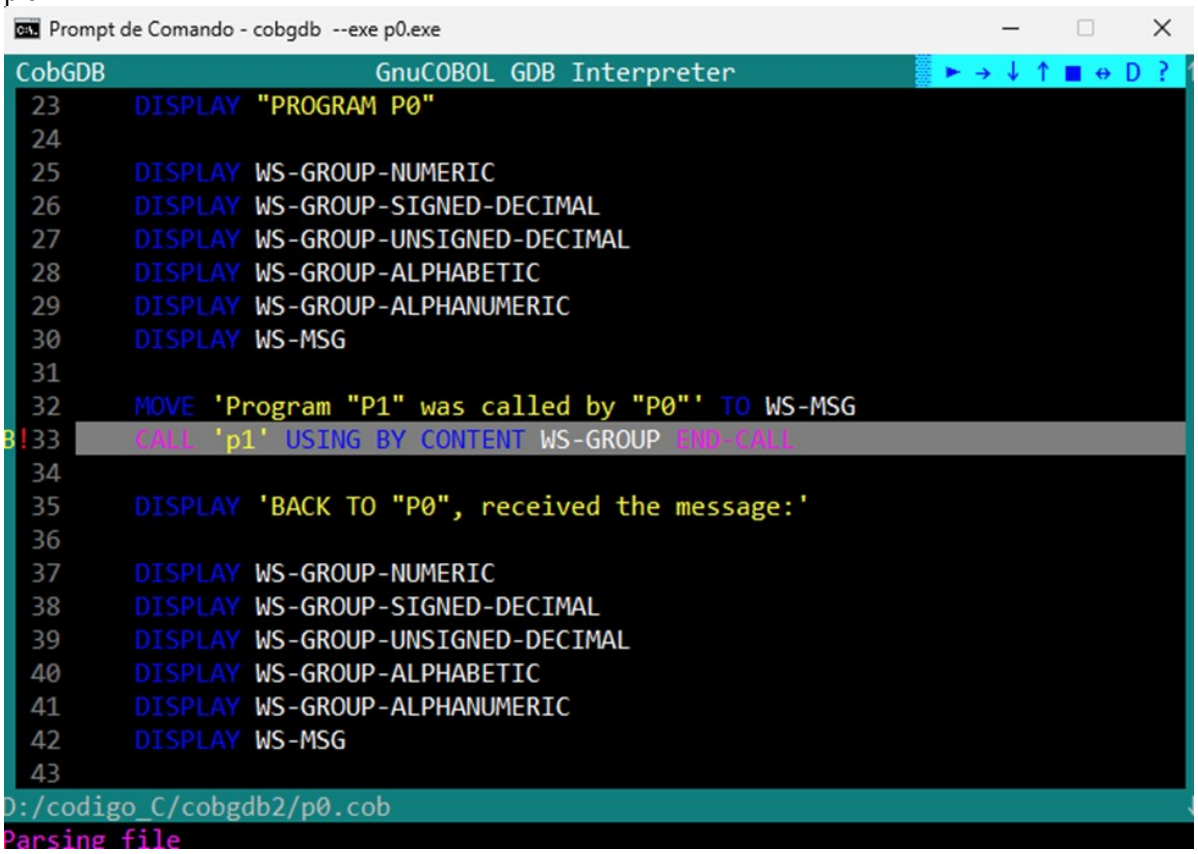
It is possible to debug modules as long as you compile them in the same directory as your source code, using the following command:

```
cobc -g -fsource-location -ftraceall -v -O0 -x p0.cob
cobc -g -fsource-location -ftraceall -v -O0 -m p1.cob
```

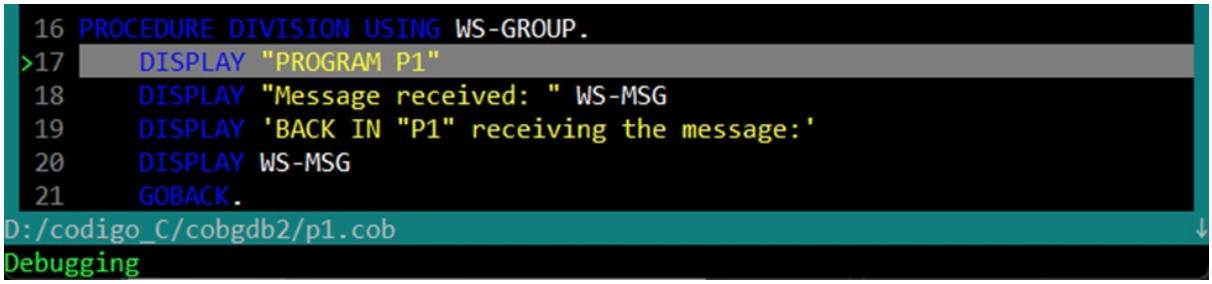
In this example, p0.cob is the source code of the executable, and p1.cob is the source code of the module (.dll on Windows or .so on Linux). Debug Execution:

```
cobgdb --exe p0.exe
```

Example:



Resulting in:



If p1.cob is already compiled as a module with:

```
cobc -g -fsource-location -ftraceall -v -O0 -m p1.cob
```

the following command also works:

```
cobgdb p0.cob
```

When you are debugging separately compiled subprograms, as shown in the previous chapter, you can use the same **F File** command, which you should use when debugging subprograms that have all been compiled into the same executable. With **F File** command you can load the source code of a module and set a breakpoint.

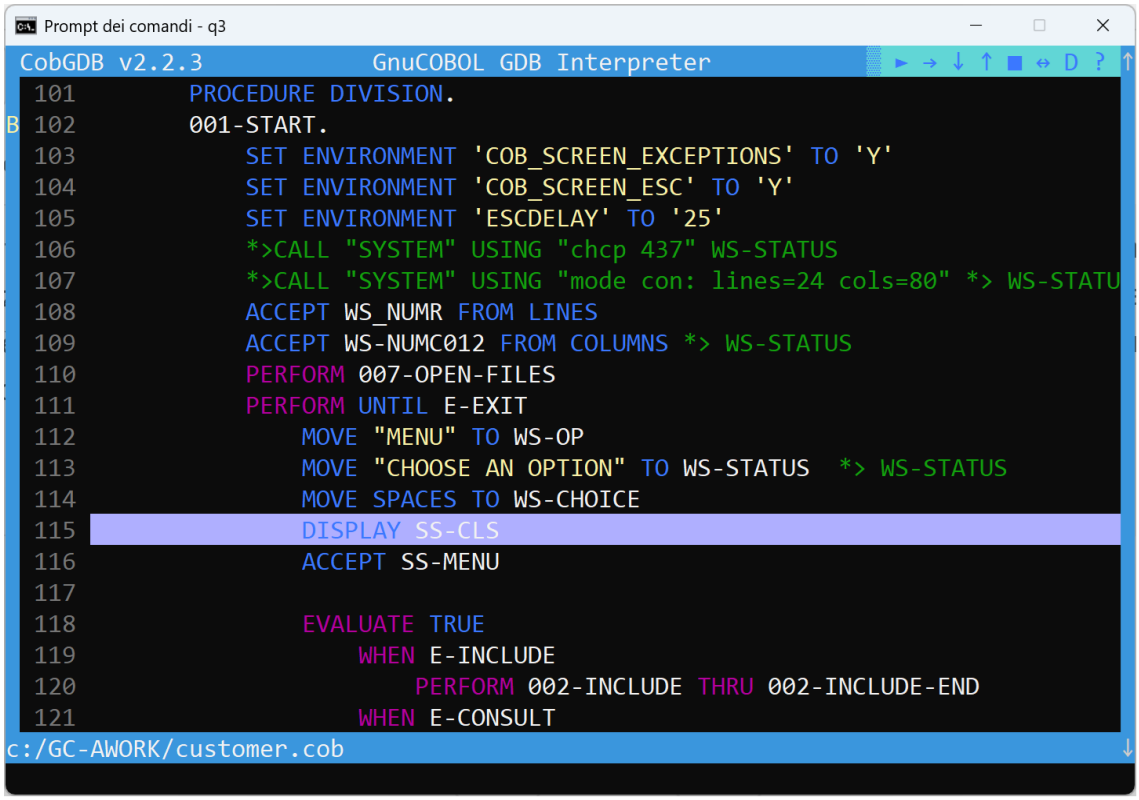
3.8. CobGDB Version

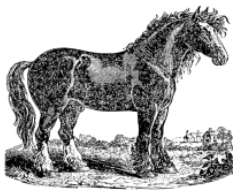
Use command option: **`cobgdb --version`**

to display CobGDB version informations as follows:

```
c:\GC-AWORK>cobgdb --version
CobGDB - GnuCobol GDB Interpreter - version 2.2.1
Copyright (C) 2013 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
This CobGDB was configured as "MinGW32".
For bug reporting instructions, please see:
<https://github.com/marcsosduma/cobgdb>.
```

Note: from version 2.2.3 you can also see the version number also in the main CobGDB screen at top left:



4. Document Change Log

CHANGE LOG
Version 1 of 2023.12.12. *First release* *. **Version 2 of 2023.12.23.*** *Step by Step sample of use is added* *Some minor changes* *. **Version 3 of 2024.02.18.*** *Restructured showing new cobgdb screens and features* *. **Version 4 of 2024.04.01 and 20240403.*** *Added EDIT subcommand at H Show Command when viewing the variable from a line of code* *Added cobgdb --version option* *. **Version 5 of 2024.05.01.*** *Added the W Window size command to change User interface screen size* *. **Version 6 of 2024.05.05.*** *Added the --exe option to debug precompiled program* *. **Version 7 of 2025.04.24.*** *Added the new D command and icon useful to enable or disable automatic display variables during debugging / animation* *. **Version 8 of 2025.06.16.*** *Added the new O (Focus) command and icon useful to switch the focus to application screen from debugging / animation screen* *. **Version 9 of 2025.10.20.*** *Added more documentation for the A (Attach) command.* ***Version 10 of 2025.11.06.*** *Added "Debugging sub programs" Chapter.* *Added F = File command also for manage breakpoints for separately compiled subprograms* *Added CTRL-F and CTR-G commands* ***Version 11 of 2025.12.01.*** *Added the Windows 11 tip to change to Windows Console Host for command W* *Added CTRL-S, CTRL-B and + / - commands* *Added “mouse hover” on a variable name feature* ***Version 12 of 2026.03.25.*** *CobGDB version number on program screen* *Added J Jump command* *Scroll symbols, scroll options on source listing* ***Version 13 of 2026.04.30.*** *Added CTRL-A Autostep command*